Welcome to the osu!framework wiki!

Disclaimer

This wiki is still unfinished so expect broken links, but of course, you can help!

This page and the rest of the wiki pages are ported from osu!framework's GitHub wiki page.

Do note that this page is currently **UNOFFICIAL** and may have outdated informations.

---

osu!framework is a framework with rhythm games such as osu! in mind.

This framework is intended to take steps beyond what you would normally expect from a game framework. This means things like basic UI elements, text rendering, advanced input handling (textboxes) and performance overlays are provided out-of-the-box!

There are multiple projects that have used osu!framework:

- osu! – rhythm is just a *click* away! (obviously)
- GDEdit - A third-party Geometry Dash editor.
- Vignette - An OpenCV-based facial recognition software for Live2D
- IWBTM - A platform game with level editor based off of "I Wanna..." games
- DeltaDash - A multi-direction, lane-based scroller rhythm game
- fluXis - A community-driven rhythm game with a focus on creativity and expression

This wiki will give you a rough guide on giving you a head start on making your first project in osu!framework!

Setting up your First Project

This tutorial will show you how to create a new project integrating osu-framework from scratch. It is not meant to be a guide for contributing to the osu-framework project itself.

Getting started

Creating a new project

To get started, a custom osu! game base project has been made available as a template pack in the .NET command line interface. Using this template, it is possible to quickly and easily generate a new project using `osu-framework` as a starting point for creating your own game.

1. In your command line utility of choice, navigate to the folder in which you want to create the project.
2. Make sure the `osu-framework` project template is downloaded by running `dotnet new install ppy.osu.Framework.Templates`.
3. Run `dotnet new osu-framework-game -n <MyNewProjectName>` to generate a new folder containing the project.

   > **Warning** Do not use spaces or hyphens in your project name. This does not play nice with the templating system.

4. Open the folder, and open `MyNewProjectName.sln` to begin!

> In addition to the `osu-framework-game` template, a template named `osu-framework-flappy-game` is also available which serves as an example of creating a basic, but feature complete game with `osu-framework`.

Project structure

When you open the solution file, you'll notice that there are three build projects inside it:

- `MyNewProject.Game`: The main project that integrates with `osu-framework` and allows you to extend it by adding your own classes. All of your game specific logic should be implemented in this project.
- `MyNewProject.Desktop`: As opposed to mobile devices, the `Desktop` project provides all of the system mechanisms and resources necessary to successfully run the project on desktop platforms. Separating this project from the `Game` project allows flexibility in supporting multiple platforms, such as mobile.

- `MyNewProject.Game.Tests`: This project allows you to write and test the logic and UI of your project. It allows access to the Test Browser, which is a separate visual testing framework that will allow you to test your UI elements and other visuals.

Game logic

If you open up `MyNewProjectName.Game/MyNewProjectNameGame.cs`, you'll see the class responsible for some basic game logic. It inherits from the `osu.Framework.Game` super-class and by default, implements a basic screen stack setup:

```
namespace MyNewProjectName.Game
{
    public partial class MyNewProjectNameGame : MyNewProjectNameGameBase
    {
        private ScreenStack screenStack;

        [BackgroundDependencyLoader]
        private void load()
        {
            // Add your top-level game components here.
            // A screen stack and sample screen has been provided for convenience, but you
can replace it if you don't want to use screens.
            Child = screenStack = new ScreenStack { RelativeSizeAxes = Axes.Both };
        }

        protected override void LoadComplete()
        {
            base.LoadComplete();

            screenStack.Push(new MainScreen());
        }
    }
}
```

The main (and only) screen contains a spinning box visual:

```
namespace MyNewProjectName.Game
{
    public partial class MainScreen : Screen
    {
        [BackgroundDependencyLoader]
        private void load()
        {
            InternalChildren = new Drawable[]
```

```
            {
                new Box
                {
                    Colour = Color4.Violet,
                    RelativeSizeAxes = Axes.Both,
                },
                new SpriteText
                {
                    Y = 20,
                    Text = "Main Screen",
                    Anchor = Anchor.TopCentre,
                    Origin = Anchor.TopCentre,
                    Font = FontUsage.Default.With(size: 40)
                },
                new SpinningBox
                {
                    Anchor = Anchor.Centre,
                }
            };
        }
    }
}
```

Platform specific logic

The logic needed to begin execution of the game is located in `MyNewProjectName.Desktop/Program.cs`:

```
namespace MyNewProjectName.Desktop
{
    public static class Program
    {
        public static void Main()
        {
            using (GameHost host = Host.GetSuitableDesktopHost(@"MyNewProjectName"))
            using (osu.Framework.Game game = new MyNewProjectNameGame())
                host.Run(game);
        }
    }
}
```

Testing

The test browser

`osu-framework` includes a visual testing framework that helps provide tests that can be verified both visually and systematically via NUnit. When running the tests project, a utility called the **test browser** will collect all of your tests, and present them in an executable that presents all of the tests in a list.

The pre-generated project will feature an example test class in `MyNewProjectName.Game.Tests/Visual/TestSceneMyNewProjectNameGame.cs`.

In order for your test browser to discover tests, you will need to specify a namespace in which for the browser to look in when constructing it:

```
    public partial class MyNewProjectNameTestBrowser : MyNewProjectNameGameBase
    {
        protected override void LoadComplete()
        {
            base.LoadComplete();

            AddRange(new Drawable[]
            {
                new TestBrowser("MyNewProjectName"), // <- see namespace specification here
                new CursorContainer()
            });
        }

        // rest of class omitted for brevity
    }
}
```

Below is an example of a visual test that displays `MyNewProjectNameGame` in full inside the test browser:

```
namespace MyNewProject.Game.Tests.Visual
{
    public partial class TestSceneMyNewProjectNameGame : TestScene
    {
        private MyNewProjectNameGame game;

        [BackgroundDependencyLoader]
        private void load(GameHost host)
        {
            game = new MyNewProjectNameGame();
            game.SetHost(host);

            AddGame(game);
        }
```

```
    }
}
```

Note that in order for you to be able to run the visual tests, you will have to switch your run configuration to the visual tests project. The solution template will generate the setup code necessary for this in `MyNewProjectName.Game.Tests/Program.cs`.

```
namespace MyNewProjectName.Game.Tests
{
    public static class Program
    {
        public static void Main()
        {
            using (GameHost host = Host.GetSuitableDesktopHost("visual-tests"))
            using (var game = new MyNewProjectNameTestBrowser())
                host.Run(game);
        }
    }
}
```

Adding tests to the Test Browser

Now that our Test Browser is discovering tests from the specified namespace, we can start adding tests! To do so, create a new class that derives `TestScene` with the `TestFixture` attribute. From here, we can add steps to this test of various types. For information on what types of steps are available, please refer to Dynamic Compilation and Visual Testing.

Example Test

The following code adds a simple cube to the visual test browser that we created above. The cube has a rigid body attached, and should drop to the bottom of the screen when created. From here, we can choose to check the behavior of the cube by asserting that the cube eventually reaches the bottom via `AddAssert()`.

```
namespace AwesomeGame.VisualTests
{
    [TestFixture]
    public class RigidCubeTest : TestScene
    {
        private RigidBodySimulation sim = null!;

        [BackgroundDependencyLoader]
```

```
        private void load()
        {
             // Set up the simulation once before any tests are ran
             Child = sim = new RigidBodySimulation { RelativeSizeAxes = Axes.Both };
        }

        [Test]
        public void AwesomeTestName()
        {
            AddStep("Drop a cube", performDropCube);
        }

        private void performDropCube()
        {
            // Add a new cube to the simulation
            RigidBodyContainer<Drawable> rbc = new RigidBodyContainer<Drawable>
            {
                Child = new Box
                {
                    Anchor = Anchor.Centre,
                    Origin = Anchor.Centre,
                    Size = new Vector2(150, 150),
                    Colour = Color4.Tomato,
                },
                Position = new Vector2(500, 500),
                Size = new Vector2(200, 200),
                Rotation =  45,
                Colour = Color4.Tomato,
                Masking = true,
            };

            sim.Add(rbc);
        }
    }
}
```

Appendix

`SetUp` attribute

The `SetUp` NUnit attribute marks a method as a setup method that runs as a step before every group of tests in a test method. The steps created by this attribute get added to the visual test browser as well.

`BackgroundDependencyLoader` attribute

The `BackgroundDependencyLoader` attribute denotes a method to be the load method of a Drawable. You can specify a type in the method parameters to attempt to grab an object of that type that has been cached.

Further reading

- For information on how to load your own resources such as textures and audio, please read Setting Up Compiled Resource Stores.
- For more information regarding dependency injection via the BackgroundDependencyLoader attribute, please read Dependency Injection
- For additional reading on visual tests, please refer to Dynamic Compilation and Visual Testing

osu!framework has framework key bindings in `FrameworkActionContainer` for specific actions listed below: | Key binding | Framework action | |:-----------:|:-------------------- | | `Ctrl` + `F1` | Toggles the draw visualiser. | | `Ctrl` + `F2` | Toggles the global statistics (and enabled performance logging while visible). | | `Ctrl` + `F3` | Toggles the texture atlas viewer. | | `Ctrl` + `F7` | Cycle frame limiter modes. | | `Ctrl` + `Alt` + `F7` | Cycle threading execution modes (single-/multithreaded). | | `Ctrl` + `F9` | Toggles the audio mixer viewer. | | `Ctrl` + `F10` | Toggles the log overlay. | | `Ctrl` + `F11` | Cycles through the frame statistics view states. (`Minimized`, `Extended`) | | `Alt` + `Enter` / `F11` | Cycles through the game window view modes. (`Windowed`, `Fullscreen`, `Borderless`) |

In addition, `TestBrowser` implementation also allow for the following key bindings:

Key binding	Action
`Ctrl` + `H`	Hide the tests list sidebar
`Ctrl` + `F` / `Cmd` + `F`	Focus the search box
`Ctrl` + `R` / `Cmd` + `R`	Reload the currently displayed test scene

Setting up Compiled Resource Stores

This tutorial assumes you have completed setting up your first project.

Adding your resources library to the Resource Store

The first time you set up your project, you will need to add a `DllResourceStore` with the path to a resources library. This will enable you to load resources from the specified resource library at run-time, which then in turn allows caching to occur in the `ResourceStore`. A few important resource stores are already created in the base `Game` class which all game instances should inherit.

By default, your resources will be compiled into a dll with your project name and copied into your output directory. This is convenient for our purposes, as all we need to do now is add a `DllResourceStore` to it using `Resources.AddStore()` in our `BackgroundDependencyLoader` method inside of our game class.

```
[BackgroundDependencyLoader]
private void load()
{
     Resources.AddStore(new DllResourceStore(@"AwesomeGame.dll"));
}
```

Default resources structure and adding new resources

`Game` will automatically create the following stores that point to the respective paths for each store:

- `/Textures` (`TextureStore`)
- `/Tracks` (`AudioManager.Tracks`)
- `/Samples` (`AudioManager.Samples`)
- `/Shaders` (`ShaderManager`)
- `/Fonts` (`FontStore`)

To add to these stores, simply add resources to the respective directories and specify them as `EmbeddedResources` inside your `.csproj`.

For example, to include your own Textures folder and all .png files inside of it recursively:

```
<ItemGroup>
 <EmbeddedResource Include="Textures\**\*.png" />
```

```
</ItemGroup>
```

Adding custom fonts to the font store

osu!framework loads Open Sans into the FontStore by default. If you find that this is insufficient for your use, you may add your own fonts to the `FontStore` by adding your own GlyphStores. This is preferably done at the initialization of your game within the `BackgroundDependencyLoader` method inside your game class by invoking the following:

```
Fonts.AddStore(new GlyphStore(Resources, @"Fonts/AwesomeFont"));
```

This will cache our hypothetical "AwesomeFont" for use in any SpriteText, which can be set using the `Font` property.

Accessing resources from the resource store

After having added your own resources, you can now access them via dependency injection. You will need to specify the type of store you're trying to inject in the method parameters. For example, if I have a texture named `awesomeTexture.png` inside of my `Textures` folder, I can load it like this:

```
[BackgroundDependencyLoader]
private void load(TextureStore store)
{
   texture = store.Get(@"awesomeTexture");
}
```

osu!framework will then load the resources for the first time if they have not been cached yet.

To retrieve other resources from other resource stores, simply specify the type of resource store you wish to retrieve from the cache via the load() method's parameters. For example, to retrieve tracks from the audio store, the following code will be sufficient.

```
ITrackStore tracks = null!;

[BackgroundDependencyLoader]
private void load(AudioManager audio)
{
   tracks = audio.Tracks;
}
```

Setting up Key Binding Container

Once you have completed setting up your first project, we are ready to start building our game! In addition to having the ability to handle input events of various types, you can also set up a `KeyBindingContainer` which allows you to specify default bindings for actions, as well as choose which key handling mode should be used in the container.

Creating a new `KeyBindingContainer`

Create an enum for our custom actions

First, we should name every input we want to be a part of this `KeyBindingContainer`. This lets us use the type field when inheriting the `KeyBindingContainer<T>` class so that it knows what action data it should map input actions to.

For this example, we will map our `W`, `A`, and `D` keys to our Jump, Left, and Right actions respectively. We will create our enum type like so:

```
public enum InputAction
{
    Left,
    Right,
    Jump
}
```

It is recommended you create this new type inside the same file where you're creating the new `KeyBindingContainer` itself in step 2. That way, it will be easier to keep track of where it is if someone were to try to find them, as well as guarantee that both exist in the same namespace.

Create a `KeyBindingContainer` class

We will then take the enum type we just created and use it to create our KeyBindingContainer.

```
public partial class AwesomeKeyBindingContainer : KeyBindingContainer<InputAction>
{
}
```

Create default keybindings for our KeyBindingContainer

We can now map key presses to the action we wish to perform! To do so, we should first create a default set of keybinds. In our newly created KeyBindingContainer, override the existing `DefaultKeyBindings` property with your own:

```
public partial class AwesomeKeyBindingContainer : KeyBindingContainer<InputAction>
{
  public override IEnumerable<KeyBinding> DefaultKeyBindings => new[]
  {
     new KeyBinding(new[] { InputKey.A }, InputAction.Left),
     new KeyBinding(new[] { InputKey.W }, InputAction.Jump),
     new KeyBinding(new[] { InputKey.D }, InputAction.Right)
  }
}
```

If you wish to have multiple alternative key combinations, you will need to specify multiple `KeyBinding` instances associated with a common `InputAction`.

Key Binding Modes

There are two modes that can be specified on the creation of a new keybinding container via the parameters. One is the `SimultaneousBindingMode`, and the other is the `KeyCombinationMatchingMode`. The `SimultaneousBindingMode` specifies whether or not an input should be valid based on whether or not other buttons are pressed simultaneously, whereas the `KeyCombinationMatchingMode` specifies how strict the keybinding should be in regards to whether or not the exact buttons are pressed.

For a better visual representation of these modes and how they restrict multiple key press events, please see `TestCaseKeyBindingsGrid`.

Handling key bound actions

Now that we have created our `KeyBindingContainer`, we can add behavior that triggers when the key-bound action has been fired. The `IKeyBindingHandler<T>` interface provides a way for bound input events to be handled by any drawable, as long as it is a child of our `KeyBindingContainer`.

For example, to create a player class that inherits `RigidBodyContainer` and implements `IKeybindingHandler`, we would do this:

```
public partial class Player : RigidBodyContainer<Drawable>, IKeyBindingHandler<InputAction>
{
}
```

The `IKeyBindingHandler` interface provides access to the following:

- `OnPressed(KeyBindingPressEvent<T>)` is triggered when any action is pressed. The action type depends on the type specified in the type specifier when you inherit `IKeyBindingHandler<T>`. Return `true` if this action should block the action from traversing the scene graph up to its parent.

- `OnReleased(KeyBindingReleaseEvent<T>)` is triggered when any action is released. This handler cannot be blocked, and will always be fired on the drawable that handled the previous press.

Example

Our completed player `RigidBodyContainer` with keybinding handling would then look something like this:

```
public partial class Player : RigidBodyContainer<Drawable>, IKeyBindingHandler<InputAction>
{
    private float constantXForce;

    [BackgroundDependencyLoader]
    private void load(TextureStore textureStore)
    {
        Child = new Box
        {
            Anchor = Anchor.Centre,
            Origin = Anchor.Centre,
            Size = new Vector2(150, 150),
        };
        Position = new Vector2(500, 500);
        Size = new Vector2(200, 200);
        Rotation = 45;
        Masking = true;
    }

    private const float PLAYER_VELOCITY = 500f;

    public bool OnPressed(KeyBindingPressEvent<InputAction> e)
    {
        switch (e.Action)
        {
            case InputAction.Jump:
                Velocity = new Vector2(constantXForce, Velocity.Y - 500);
                break;

            case InputAction.Right:
                constantXForce += PLAYER_VELOCITY;
                break;

            case InputAction.Left:
                constantXForce -= PLAYER_VELOCITY;
                break;
        }

        return true;
```

```
    }

    public void OnReleased(KeyBindingReleaseEvent<InputAction> e)
    {
        switch (e.Action)
        {
            case InputAction.Left:
                constantXForce += PLAYER_VELOCITY;
                break;

            case InputAction.Right:
                constantXForce -= PLAYER_VELOCITY;
                break;
        }
    }

    protected override void Update()
    {
        base.Update();

        Velocity = new Vector2(constantXForce, Velocity.Y);
    }
}
```

For this example to work as intended, we will need to change the simultaneous binding mode in our keybinding container to allow multiple bindings to be pressed at the same time. We can specify a constructor for this:

```
public AwesomeKeyBindingContainer(KeyCombinationMatchingMode keyCombinationMatchingMode
= KeyCombinationMatchingMode.Any, SimultaneousBindingMode simultaneousBindingMode
= SimultaneousBindingMode.All)
    : base(simultaneousBindingMode, keyCombinationMatchingMode)
{
}
```

Setting up Fonts

This tutorial assumes you have completed setting up your first project and setting up compiled resource stores.

In this tutorial, you will learn how to add custom fonts into your game and use it:

- Converting to binary font and texture files
- Adding the custom font to the font store
- Using specific font for sprite text

Converting to binary font and texture files

For the custom font to be recognized inside osu!framework, you will have to convert it into a binary font with texture files alongside it.

To start off, you'll need to download and install BMFont by AngelCode for converting the font, then run it and load this configuration for selecting required character range:

 **Load the required configurations #1**	 **Load the required configurations #2**

Next, you have to set up the tool into using and converting your custom font properly by going into the font settings and selecting your custom font and check/uncheck `Bold` and `Italic`, and going into the export settings and selecting `Binary` as the font descriptor. **Without it, your game might hard crash when loading the font.** |  Set up tool settings #1 | Set up tool settings #2 | |---|---| | Set up tool settings #3 | Set up tool settings #4 |

And lastly, converting the font can be accomplished by clicking on `Save bitmap font as` and saving it wherever you want inside your `Resources` folder, which then will be pointed out to later on: |  Save binary font #1 |  Save binary font #2 | |---|---|

The font file name should be saved in the following pattern: | `FontUsage.Family` | `FontUsage.Weight` | `FontUsage.Italics` | Font file name | |----------|:----------:|:----------:|:----------| | MyAwesomeFont | `"Light"` | `false` | `MyAwesomeFont-Light` | | MyAwesomeFont | `"Bold"` | `true` | `MyAwesomeFont-BoldItalic` | | MyAwesomeFont | `null` | `false` | `MyAwesomeFont` | | MyAwesomeFont | `null` | `true` | `MyAwesomeFont-Italic` |

Adding the custom font to the font store

To use the custom font inside your game, you'll need to add it to the `FontStore` by specifying:

- The resource store of your resources folder that contains the binary fonts.
- The asset name of the binary font, The font path should be specified here relative from the resource store you've specified. (`Fonts/MyAwesomeFont` for this tutorial)

Here's an example of adding a custom font included within a `Resources` folder inside the game project: (`MyAwesomeProject.Game/Resources/Fonts/MyAwesomeFont`)

```
namespace MyAwesomeProject.Game
{
    public class MyAwesomeGame : osu.Framework.Game
    {
        [BackgroundDependencyLoader]
        private void load()
        {
            // Add the DLL resource store of this game project into the global
resources store.
            Resources.AddStore(new DllResourceStore(@"MyAwesomeProject.dll"));

            // Add the custom font to the global font store (Fonts).
            AddFont(Resources, @"Fonts/MyAwesomeFont");
        }
    }
}
```

If you have specified a custom size for the font you've exported, you'll need to apply scale adjustments for its font store to match text sizing with different added fonts, can be done by:

```
namespace MyAwesomeProject.Game
{
    public class MyAwesomeGame : osu.Framework.Game
    {
        [BackgroundDependencyLoader]
        private void load()
        {
            // Add the DLL resource store of this game project into the global
resources store.
            Resources.AddStore(new DllResourceStore(@"MyAwesomeProject.dll"));

            // Create a font store with providing a specific font size (200px for
this example)
            // to adjust scaling for matching up with different fonts.
            var scaleAdjustedFontStore = new FontStore(scaleAdjust: 200);

            // Nest the scale-adjusted font store inside the global font store
```

```
            // for components such as sprite texts be able to retrieve the custom font.
            Fonts.AddStore(scaleAdjustedFontStore);

            // Add fonts of a same custom size (200px for this example) to the scale-
adjusted font store.
            AddFont(Resources, @"Fonts/MyAwesomeFont", scaleAdjustedFontStore);
        }
    }
}
```

> **Warning:** Certain font names may end up getting mangled by the .NET embedded resource machinery. For instance, numbers may be prefixed with underscores. See https://stackoverflow.com/questions/14705211/how-is-net-renaming-my-embedded-resources for more detail. A safe bet is to stick with letters.

Now you're almost done, by default, the first font to be added to your game will automatically be used for all `SpriteText`s in your game, but you may want to use a specific font.

Using specific font for a sprite text

To use a specific font for a sprite text, all you have to do is construct a new `FontUsage` with: | Parameter | Description | Value in this tutorial | | --------- | ----------- | ---------------------- | | `family` | The font name | `MyAwesomeFont` | | `weight` | The weight of the font in string | `null` | | `italics` | Whether the font is of italic type | `false` |

Check the font file name table example above for learning more.

There are also other `FontUsage` properties you can modify for rendering the font differently (e.g. `size` and `fixedWidth`)

This opens up for a lot of ways to implement the usage of your custom font, either by adding static properties for constructing the `FontUsage` or adding a static function with enumeration parameters that you can pick your font and its properties from, all based on your preference.

Here's an example of creating a sprite text using the custom font with `size: 40f`: | Code | Preview |---|---|

Breaking changes

Occasionally we will make changes which require consumers of the framework to make amendments to maintain compatibility. Wherever possible, we maintain compatibility via `[Obsolete]` attributes, but it is encouraged that you leave obsolete warnings turned on, and deal with them sooner rather than later. Generally we aim to leave obsolete methods around for 3-6 months after obsoletion.

This page serves to give a list of all breaking/major changes.

2024.121.1

`DrawablePool.Get()` wil now throw if the `DrawablePool` did not begin load

Previously the pool would return drawables correctly before load, but it would not initialise the pool to correct default size, causing overheads on drawable retrieval (usually on update thread).

`HostOptions.BindIPC` has been superseded by `HostOptions.IPCPort`

Previously, the IPC port used for multiple instances of a single osu!framework app was hardcoded in the IPC host itself, effectively making it so that *all osu!framework apps* would share the same IPC port, which obviously cannot work.

To allow multiple osu!framework apps to utilise IPC concurrently, `BindIPC` has thus been replaced by `IPCPort`.

- Setting `IPCPort = null` is equivalent to `BindIPC = false`.
- Setting `IPCPort` to a non-null value is equivalent to `BindIPC = true` *and* will force the use the specific port provided.

Note that it is advised to use a "user port" (in the range of 1024-49151) as per RFC 6335.

2023.1213.0

`IParseable.Parse()` has received an `IFormatProvider` argument

This is provided to better treat input when typed by end users, by allowing the ability to respect their regional number settings.

Existing usages of `IParseable.Parse()` should pass `CultureInfo.InvariantCulture` to preserve behaviour, including usages of `IBindable.Parse()`:

```
  Bindable<int> bindable = new Bindable<int>();
- bindable.Parse(5);
+ bindable.Parse(5, CultureInfo.InvariantCulture);
```

`DrawNode.Draw()` is no longer exposed publicly

`Draw()` is now `protected` to match the signature of `DrawOpaqueInternal()`. `DrawNode`s which draw others no need to do so via a separate (provided) method.

```
- public override void Draw(IRenderer renderer)
+ protected override void Draw(IRenderer renderer)
  {
      base.Draw(renderer);

-     other.Draw(renderer);
+     DrawOther(other, renderer);
  }
```

`Dropdown` requires a search bar to be implemented

All `Dropdown`s may now be searched, which requires them to implement a search bar component. Sample implementation:

```
public partial class MyDropdownHeader : DropdownHeader
{
    protected override DropdownSearchBar CreateSearchBar() => new MyDropdownSearchBar();

    public partial class MyDropdownSearchBar : DropdownSearchBar
    {
        protected override void PopIn() => this.FadeIn();

        protected override void PopOut() => this.FadeOut();

        protected override TextBox CreateTextBox() => new MyTextBox
        {
            PlaceholderText = "type to search"
        };
    }
}
```

2023.1004.1

`DecoupleableInterpolatingFramedClock` is obsoleted. Use `DecouplingClock` instead

See this PR for more details on a migration path.

2023.914.0

Masking-related breaking changes from 2023.822.0 have been reverted

After pushing the SSBO masking changes to users, it turned out that the change was not an unambiguous performance win as originally hoped, and as such has been reverted for now in order to avoid diverting focus further away from more important concerns.

In response, framework consumers need to revert any and all changes incurred by the aforementioned release. It is advised to keep the changes somewhere around, however, as the SSBO concept may be revisited at a later date.

2023.904.0

`Storage.Move()` will overwrite file at target destination if present

Previously it would fail if a file already existed at the target destination. Overwriting is usually the preferred outcome.

2023.822.0

Maskable vertex input layout has changed

Vertices which want to use the built-in `VertexShaderDescriptor.TEXTURE` fragment shader, or the `sh_Masking.h` particle, will need to adjust the vertex input slightly:

```
+ #include "Internal/sh_MaskingInfo.h"

layout(location = 0) in vec2 m_Position;
+ layout(location = 1) in int m_MaskingIndex;

+ layout(location = 5) flat out int v_MaskingIndex;
+ layout(location = 6) out highp vec2 v_ScissorPosition;
```

```
 void main(void)
 {
+    InitMasking(m_MaskingIndex);

     // Transform from screen space to masking space.
-    highp vec3 maskingPos = g_ToMaskingSpace * vec3(m_Position, 1.0);
+    highp vec4 maskingPos = g_MaskingInfo.ToMaskingSpace * vec4(m_Position, 1.0, 0.0);
     v_MaskingPosition = maskingPos.xy / maskingPos.z;

+    // Transform from screen space to scissor space.
+    highp vec4 scissorPos = g_MaskingInfo.ToScissorSpace * vec4(m_Position, 1.0, 0.0);
+    v_ScissorPosition = scissorPos.xy / scissorPos.z;

+    v_MaskingIndex = m_MaskingIndex;

     gl_Position = g_ProjMatrix * vec4(m_Position, 1.0, 1.0);
 }
```

Likewise, the C# definition must also be updated to write the `m_MaskingIndex` input, and be constructed with an `IRenderer`:

```
 [StructLayout(LayoutKind.Sequential)]
 public struct MyCustomVertex : IEquatable<MyCustomVertex>, IVertex
 {
     [VertexMember(2, VertexAttribPointerType.Float)]
     public Vector2 Position;

+    [VertexMember(1, VertexAttribPointerType.Int)]
+    private readonly int maskingIndex;

+    public MyCustomVertex(IRenderer renderer)
+    {
+        this = default;
+        maskingIndex = renderer.CurrentMaskingIndex;
+    }

     public readonly bool Equals(MyCustomVertex other) =>
         Position.Equals(other.Position)
+        && maskingIndex == other.maskingIndex;
 }
```

Non-masking fragment shaders must use non-masking vertex shaders

Any usages of `VertexShaderDescriptor.TEXTURE_2` as a vertex shader in-combination with a non-masking fragment shader (i.e. one that does not use the built-in `sh_Masking.h` particle), should be updated to use `VertexShaderDescriptor.TEXTURE_2_NO_MASKING` instead.

`IRenderer.PushScissorOffset()`/`IRenderer.PopScissorOffset()` have been removed

The way masking applies scissor has changed such that this no longer has any use.

`MaskingInfo` layout has changed

- `ScreenSpaceAABB` renamed to `ScreenSpaceScissorArea`.
- `MaskingRect` renamed to `MaskingArea`
- `ToScissorSpace` added. If `ScreenSpaceScissorArea` truly represents a screen-space area, set this to `Matrix3.Identity`, otherwise set it to convert vertex coordinate inputs to the appropriate coordinate space of `ScreenSpaceScissorArea`.

2023.817.0

`TexturedVertex2D` must be initialised with an `IRenderer`.

The default constructor for this type has been obsoleted and will now generate a compiler error.

2023.720.0

Source generators will now only run on release builds

As we continue to add more source generators, we've seen increases in compile-time overheads, with local testing showing over 2x compile times with source generators turned on.

osu!framework source generators are made to optimise builds at runtime (mostly by removing reflection overhead). As such, it doesn't make sense to run these for debug releases as they are basically custom release-targeting optimisations.

This should not result in a noticeable change in runtime performance during debug, but will reduce compilation times by over 50% in most cases. This is valuable during debug as the most common case is frequently building / hot reloading for quick iteration.

2023.714.0

`GameHost.GetClipboard()` is obsolete

The new way to retrieve a `Clipboard` instance is to resolve it via dependency injection directly. This does not require resolving the `GameHost` anymore:

```
 [BackgroundDependencyLoader]
-private void load(GameHost host)
+private void load(Clipboard clipboard)
 {
-    host?.Clipboard.SetText("example");
+    clipboard.SetText("example");
 }
```

2023.618.0

`AddDropdownItem(LocalisableString text, T value)` has been removed

If you were using this, refactor your code to use `LocalisableString GenerateItemText(T value)` instead.

Easiest way is to add `LocalisableString? CustomDropdownText` to your `T`, or wrap `T` in a class that has one. Then simply change `GenerateItemText()` to use that:

```
public partial class MyDropdown : Dropdown<T>
{
    protected override LocalisableString GenerateItemText(T value)
        => value.CustomDropdownText ?? base.GenerateItemText(value);
}
```

Subclasses of `FocusedOverlayContainer` must now implement `Pop{In,Out}()`

Previously, subclasses of `FocusedOverlayContainer` were not forced to implement `Pop{In,Out}` themselves, despite actually needing to do so for the overlay to actually play transitions out correctly. This has now been changed and inheritors of the class must always implement `Pop{In,Out}`.

In practice this should not break any reasonable consumers of the class, except for the need to remove any `base.Pop{In,Out}()` calls in classes that directly inherit `FocusedOverlayContainer` and call base in `Pop{In,Out}()`.

2023.326.0

iOS project structure has been revamped

In efforts to migrate the framework to SDL on iOS, the project structure required major changes. For consumers, you're required to remove the `AppDelegate` class attached in the game project as `GameAppDelegate` no longer exists, and change the `Application.Main` method to call `GameApplication.Main` as such:

```
-        public static void Main(string[] args) => UIApplication.Main(args,
null, typeof(AppDelegate));
+        public static void Main(string[] args) => GameApplication.Main(new MyGameIOS());
```

2023.322.0

`TexturedShaderDrawNode.TextureShader` is made private

This is partially done to force consumers into binding uniform blocks in the newly defined `TexturedShaderDrawNode.BindUniformResources` method, such that uniform blocks are always bound whenever the corresponding shader is used for drawing in the framework.

For binding/unbinding shaders, use `TexturedShaderDrawNode.BindTextureShader`/`TexturedShaderDrawNode.UnbindTextureShader` instead.

2023.314.0

The OpenGL Core profile is now used

Shaders now follow "Vulkan GLSL", which is mostly documented in the spec: GL_KHR_vulkan_glsl. The primary differences are documented below:

Vertex shader members

old	new
`attribute lowp int In_Value;` `varying lowp int Out_Value;`	`layout(location = 0) in lowp int InValue;` `layout(location = 0) out lowp int OutValue;`

The location increases for each additional member in the respective `in`/`out` lists.

Fragment shader members

old	new

`varying lowp int In_Value;`	`layout(location = 0) in lowp int InValue;`

The location increases for each additional member, and must match the location of the respective member in the vertex shader.

Fragment shader outputs

Fragment shaders are required to define an output variable.

old	new

`void main(void)`
`{`
`    gl_FragColor = vec4(0.0);`
`}`
	`layout(location = 0) out vec4 colour;`
	
	`void main(void)`
	`{`
	`    colour = vec4(0.0);`
	`}`

Uniform members

Free-floating "global" uniforms are not supported. They must be placed in "uniform blocks".

old	new

`uniform lowp int`
`Uniform_Value;`
	`layout(std140, set = 0, binding = 0)`
	`uniform MyUniforms`
	`{`
	`    lowp int Uniform_Value;`
	`};`

Notes:

- `std140` should always be used.
- The `set` increases for each unique uniform block.
- `binding` is a special required value which indicates where in the program the block should be bound. osu!framework manages this for you and should be 0 for uniform blocks.
- Both shader stages can use the same uniform block from the same set, provided that the physical layout matches.

Furthermore, this means that uniform blocks have become a first-class citizen in osu!framework. The code to make use of the above looks like this:

```
class MyDrawNode : DrawNode
{
    private IUniformBuffer<MyBufferData>? bufferData;

    public override void Draw(IRenderer renderer)
    {
        base.Draw(renderer);

        // Create the uniform buffer.
        bufferData ??= renderer.CreateUniformBuffer<MyBufferData>();

        // Update the uniform buffer.
        bufferData.Data = bufferData.Data with
        {
            UniformValue = 5
        };

        // Bind the buffer to the shader.
        shader.BindUniformBuffer("MyUniforms", bufferData);

        shader.Bind();
        // Draw...
    }

    protected override void Dispose(bool isDisposing)
    {
        base.Dispose(isDisposing);

        // Ensure that the buffer is disposed.
        bufferData?.Dispose();
    }
}

// Pack = 1 is required.
[StructLayout(LayoutKind.Sequential, Pack = 1)]
// Record structs automatically implement IEquatable<>
public record struct MyBufferData
{
    // Use types provided in the osu.Framework.Graphics.Shaders.Types namespace.
    public UniformInt UniformValue;
```

```
    // The struct must be a multiple of 16 bytes.
    private readonly UniformPadding12 pad2;
}
```

Runtime validation is present to notify of incorrect alignment/packing.

Uniform textures/samplers

The combined sampler of GLSL is not supported.

old	new
`uniform lowp sampler2D Sampler;`	`layout(set = 0, binding = 0) uniform lowp texture2D Texture;` `layout(set = 0, binding = 1) uniform lowp sampler Sampler;`

Notes:

- `std140` cannot be applied here unlike for uniform blocks.
- The `set` increases along with all other uniform blocks/textures.
- The texture and sampler are bound to different points. The texture being bound to 0 and sampler to 1 will be standard for osu!framework going forward.

Sampling textures

old	new
`vec4 col = texture2D(my_Sampler, ...);`	`vec4 col = texture(sampler2D(my_Texture, my_Sampler), ...);`

2022.1129.0

`IHasFilterableChildren` has been removed; use `IFilterable` instead (https://github.com/ppy/osu-framework/pull/5530)

Rather than rely on `IHasFilterableChildren` to descend down complex hierarchies for filtering purposes, `IContainerEnumerable<T>.Children` is used, which results in less boilerplate as you don't need to

remember to implement it whenever needed.

`IConditionalFilterable` is introduced (https://github.com/ppy/osu-framework/pull/5530)

By implementing `IConditionalFilterable`, you get one extra bindable to play with. If the value of said bindable is `true`, then the item is included in textual search as usual. If it is `false`, however, it - and its entire subtree - is excluded from further search on the premise that the item does not meet external criteria. In practical terms, you would set it to `false` when you want to hide some items inside a `SearchContainer` because they're unavailable due to other settings.

Classes used in dependency injection need to implement the `IDependencyInjectionCandidate` interface (https://github.com/ppy/osu-framework/pull/5548)

This does not affect `Drawable` subclasses.

2022.1126.0

Add source generator for dependency injection (https://github.com/ppy/osu-framework/pull/5541)

This one's been brewing for a long time while I tried to figure out the best way to do things. It provides a structural foundation on top of which we can build other source generators.

For source generators to work at all for us, we unfortunately need to make every `Drawable` class `partial`. This can be done as a graceful upgrade – previous reflection pathways are still supported so no immediate changes should be required to your project to compile.

Analyser + codefix

I've created a pretty "basic" analyser for warning against non-`partial` classes when required. It attempts to discover:

- Types used as the argument to calls of `DependencyContainer.Inject()`.
- Types used as type arguments of `CachedModelDependencyContainer<T>`.
- Types implementing `ITransformable`, `IDrawable` or `ISourceGeneratedDependencyActivator`.

This covers all current cases while being as least invasive as possible (i.e. not analysing every member of every class ever).

A code fix has been provided for the diagnostic, however if you're planning to codefix the whole solution I suggest the running the following command until it outputs 0 changes:

```
dotnet format analyzers osu.Desktop.slnf --verbosity d
```

The code fix seems to not fix all issues on the first attempt. This is potentially caused by my implementation, however I've tried everything that I could find and nothing's worked. I'll look into fixing this as a future effort, however for the most part it's a one-time thing.

The analyser and code fixer don't have tests at the moment. These are also planned in a future effort and will be structured similarly to the source generator tests and the `osu-localisation-analyser` project.

Notes

- Member accessor validation is not supported yet. I think these will probably be added to an analyser in a future effort, but I've disabled the relevant tests in https://github.com/smoogipoo/osu-framework/commit/f3324205ded5ed1c05fa3a92a8fc4b5c26ec7cb4 for now.
- Build time has increased. This can probably be improved further, however I don't want to drag on this PR much longer.
- Going to definitions of classes is a +1 process now, with every single definition now giving you two locations.
- I eventually want to update the source generator to make use of `IIncrementalGenerator`, but there's a bit more reading to do for that.
- The class is generated via AST rather than raw code. This is a personal/stylistic choice because I originally implemented it using raw code and couldn't bear it.
- The SG code quality is not quite where I want it to be at the moment - it's lacking documentation (compared to something like `osu-localisation-analyser`) and the tests are lacking structure. This will be improved in a future effort.

I have not tested VSCode compatibility, but I have no reason to believe it won't work, unlike `osu-localisation-analyser` where there's differences in how IDEs add files to workspaces from analysers/codefixes.

Performance

Check the pull request thread for CPU and memory profiling, but the short of this is that framework allocations required for dependency injection are down 50% or more; overall allocations at runtime are down 50% (by object count) and also substantially by memory; CPU overhead for DI activation (ie. creating a new `Drawable`) is down magnitudes on first usage, and also marginally on subsequent usages.

Put simply, it's a win all-round.

`Bindable<T>.CopyTo` should be implemented for any custom bindables (https://github.com/ppy/osu-

framework/pull/5531)

To improve the performance of creating clones of bindables (ie. via `GetUnboundCopy()`), the copy portion of `BindTo()` has been split out into its own method. If you have any custom bindable types which were copying values across within a `BindTo()` override, please move the copy operation to `CopyTo()` instead. Pay special attention to the direction of assignment, which will reverse from what it was in `BindTo`.

A fix should look something like this:

```
diff --git a/osu.Framework/Bindables/BindableNumber.cs
b/osu.Framework/Bindables/BindableNumber.cs
index 6d605667f..430da79d0 100644
--- a/osu.Framework/Bindables/BindableNumber.cs
+++ b/osu.Framework/Bindables/BindableNumber.cs
@@ -199,12 +199,12 @@ protected void TriggerPrecisionChange(BindableNumber<T> source = null,
bool prop
                 PrecisionChanged?.Invoke(precision);
         }

-        public override void BindTo(Bindable<T> them)
+        public override void CopyTo(Bindable<T> them)
         {
             if (them is BindableNumber<T> other)
-                Precision = other.Precision;
+                other.Precision = Precision;

-            base.BindTo(them);
+            base.CopyTo(them);
         }

         public override void UnbindEvents()
```

2022.907.0

`DepthStencilFunction` renamed to `BufferTestFunction`

The new naming should better fit future usages outside of depth and stencil tests.

2022.901.0

`CompositeDrawable.RemoveInternal`, `IContainer.Remove`, `IContainer.RemoveRange` and `IContainer.RemoveAll` now

require a `bool` parameter

A common mistake we've seen made osu!-side is where drawables are `Removed` from the hierarchy with the intention of never using them again. In such cases, disposal is not guaranteed – nor is unbinding of `Bindable` fields/properties – which can lead to event and object leakage.

To prevent this from happening, a new `disposeImmediately` parameter has been added.

Generally this should be set to `true` unless you intend to reuse the removed drawable (ie. by adding back to the hierarchy at a different location).

2022.816.0

Depth test function is now typed to `DepthStencilFunction`

In places where the `DepthInfo` struct is used with a custom depth function, the following changes are required.

```
- new DepthInfo(function: DepthFunction.Never)
+ new DepthInfo(function: DepthStencilFunction.Never)

- new DepthInfo(function: DepthFunction.Less)
+ new DepthInfo(function: DepthStencilFunction.LessThan)

- new DepthInfo(function: DepthFunction.Lequal)
+ new DepthInfo(function: DepthStencilFunction.LessThanOrEqual)

- new DepthInfo(function: DepthFunction.Equal)
+ new DepthInfo(function: DepthStencilFunction.Equal)

- new DepthInfo(function: DepthFunction.Gequal)
+ new DepthInfo(function: DepthStencilFunction.GreaterThanOrEqual)

- new DepthInfo(function: DepthFunction.Greater)
+ new DepthInfo(function: DepthStencilFunction.GreaterThan)

- new DepthInfo(function: DepthFunction.Notequal)
+ new DepthInfo(function: DepthStencilFunction.NotEqual)

- new DepthInfo(function: DepthFunction.Always)
+ new DepthInfo(function: DepthStencilFunction.Always)
```

2022.805.0

`IRenderer` added as parameter to `DrawNode`

In order to allow the framework to interoperate with different rendering backends, all rendering functions are now performed through a new `IRenderer` interface and parameter.

```
- DrawNode.Draw(Action<TexturedVertex2D> vertexAction);
+ DrawNode.Draw(IRenderer renderer);
- DrawNode.DrawOpaqueInterior(Action<TexturedVertex2D> vertexAction);
+ DrawNode.DrawOpaqueInterior(IRenderer renderer);
```

In places where the `vertexAction` parameter was previously used (e.g. to draw textures), `null` can be given as a parameter instead.

`GLWrapper` removed, replaced by calls through `IRenderer`

The static `GLWrapper` class has been removed, and all drawing functions should be performed via the new `DrawNode` parameter instead. There is a 1-1 API correlation between the classes.

```
public class MyDrawNode : DrawNode
{
    public override void Draw(IRenderer renderer)
    {
        base.Draw(renderer);

-       GLWrapper.PushDepthInfo(...);
+       renderer.PushDepthInfo(...);

-       GLWrapper.SetBlend(...);
+       renderer.SetBlend(...);

-       GLWrapper.PopDepthInfo();
+       renderer.PopDepthInfo();
    }
}
```

`Texture.WhitePixel` has moved to `IRenderer`

For cases where it's used as a fallback texture, it can be retrieved by resolving an `IRenderer` into the `Drawable` class and accessing `IRenderer.WhitePixel`.

```
public class MyDrawable : Drawable
{
+   [BackgroundDependencyLoader]
```

```
+   private void load(IRenderer renderer)
+   {
+       texture ??= renderer.WhitePixel;
+   }

    private Texture texture;

    public Texture Texture
    {
-       get => texture ?? Texture.WhitePixel;
+       get => texture;
        set => texture = value;
    }
}
```

Texture drawing methods moved from `DrawNode` to `IRenderer` extension methods

Appearing alongside `IRenderer` is the new `RendererExtensions` class providing helper methods for common drawing procedures.

```
public class MyDrawNode : DrawNode
{
    public override void Draw(IRenderer renderer)
    {
        base.Draw(renderer);

-       DrawQuad(renderer.WhitePixel, ...);
+       renderer.DrawQuad(renderer.WhitePixel, ...);
    }
}
```

All of the following methods have been moved to this class:

```
DrawTriangle()
DrawQuad()
DrawClipped()
DrawFrameBuffer()
```

Textures must be created through the `IRenderer`

Resolve an `IRenderer` into the `Drawable` class and use `IRenderer.CreateTexture()` method to create textures.

```
public class MyDrawable : Drawable
{
-   private readonly Texture texture;

-   public MyDrawable()
-   {
-       texture = new Texture(100, 100);
-       texture.SetData(...);
-   }
+   private Texture texture;

+   [BackgroundDependencyLoader]
+   private void load(IRenderer renderer)
+   {
+       texture = renderer.CreateTexture(100, 100);
+       texture.SetData(...);
+   }
}
```

`DummyRenderer` may be used for cases where textures need to be created and neither an `IRenderer` nor `GameHost` is accessible:

```
[TestFixture]
public class MyTestFixture
{
    public void CreateATexture()
    {
-       Texture texture = new Texture(1, 1);
+       Texture texture = new DummyRenderer().CreateTexture(1, 1);
    }
}
```

`TextureGL` is no longer accessible

Properties such as `TextureGL.BypassTextureUploadQueueing` have been moved to `Texture` itself, and `Texture` can be used for all rendering procedures. When rendering, textures are now bound to different sampling units via an integer value.

```
public class MyDrawable : Drawable
{
    private Texture texture;

    [BackgroundDependencyLoader]
```

```
    private void load(IRenderer renderer)
    {
        texture = renderer.CreateTexture(100, 100);
-       texture.TextureGL.BypassUploadQueueing = true;
+       texture.BypassUploadQueueing = true;
        texture.SetData(...);
    }
}

public class MyDrawNode : DrawNode
{
    private Texture texture;

    public override void Draw(IRenderer renderer)
    {
        base.Draw(renderer);

-       texture.TextureGL.Bind();
-       texture.TextureGL.Bind(TextureUnit.Texture1);
+       texture.Bind();
+       texture.Bind(1);

        // Note: The texture binding above isn't required in either case for the call below.
-       DrawQuad(texture.TextureGL, ...);
+       renderer.DrawQuad(texture, ...);
    }
}
```

The `QuadBatch<T>` and `LinearBatch<T>` classes are no longer accessible

Create vertex batches via the `IRenderer` and store as an `IVertexBatch<T>` instead:

```
public class MyDrawNode : DrawNode
{
-   private readonly QuadBatch<TexturedVertex2D> quadBatch = new QuadBatch<TexturedVertex2D>
(1, 1);
-   private readonly LinearBatch<TexturedVertex2D> linearBatch = new
LinearBatch<TexturedVertex2D>(1, 1, PrimitiveType.Triangles);
+   private IVertexBatch<TexturedVertex2D> quadBatch;
+   private IVertexBatch<TexturedVertex2D> linearBatch;

    public override void Draw(IRenderer renderer)
    {
        base.Draw(renderer);
```

```
+       quadBatch ??= renderer.CreateQuadBatch<TexturedVertex2D>(1, 1);
+       linearBatch ??= renderer.CreateLinearBatch<TexturedVertex2D>(1,
1, PrimitiveTopology.Triangles);
    }

    protected override void Dispose(bool disposing)
    {
        base.Dispose(disposing);
        batch?.Dispose();
    }
}
```

Beware that when creating linear batches, the type parameter has changed from `PrimitiveType` to `PrimitiveTopology`!

The `FrameBuffer` class is no longer accessible

Create frame buffers via the `IRenderer` and store as an `IFrameBuffer`.

```
public class MyDrawNode : DrawNode
{
-   private readonly FrameBuffer myFrameBuffer = new FrameBuffer(new[] {
RenderbufferInternalFormat.DepthComponent16 }, All.Nearest);
+   private IFrameBuffer myFrameBuffer;

    public override void Draw(IRenderer renderer)
    {
        base.Draw(renderer);

+       myFrameBuffer ??= renderer.CreateFrameBuffer(new[] { RenderBufferFormat.D16
}, TextureFilteringMode.Nearest);
    }

    protected override void Dispose(bool disposing)
    {
        base.Dispose(disposing);
        myFrameBuffer?.Dispose();
    }
}
```

Beware that the render buffer type parameter has changed from `RenderbufferInternalFormat` to `RenderBufferFormat`!

The `Shader` class is no longer accessible

Shaders are still created via the `ShaderManager`, but are now returned as `IShader`s.

```
public class MyDrawable : Drawable
{
-   private Shader shader;
+   private IShader shader;

    [BackgroundDependencyLoader]
    private void load(ShaderManager shaders)
    {
        shader = shaders.Load("a", "b");
    }
}
```

`TextureStore`, `FontStore` and `LargeTextureStore` require an `IRenderer` constructor parameter

One common use case is to provide a new texture store from inside a derived `Game`, for which the following change is required:

```
public class TestGame : osu.Framework.Game
{
    private DependencyContainer dependencies;

    protected override IReadOnlyDependencyContainer
CreateChildDependencies(IReadOnlyDependencyContainer parent) =>
        dependencies = new DependencyContainer(base.CreateChildDependencies(parent));

    [BackgroundDependencyLoader]
    private void load()
    {
-       var largeStore = new LargeTextureStore(Host.CreateTextureLoaderStore(new
NamespacedResourceStore<byte[]>(Resources, @"Textures")), All.Nearest);
+       var largeStore = new LargeTextureStore(Host.Renderer,
Host.CreateTextureLoaderStore(new NamespacedResourceStore<byte[]>(Resources,
@"Textures")), TextureFilteringMode.Nearest);
        largeStore.AddTextureSource(Host.CreateTextureLoaderStore(new OnlineStore()));
        dependencies.Cache(largeStore);
    }
}
```

Beware that the filter mode parameter has changed from `All` to `TextureFilteringMode`!

Some `TexturedShaderDrawNode` members now take an `IRenderer` parameter

```
public class MyDrawNode : TexturedShaderDrawNode
{
    public override void Draw(IRenderer renderer)
    {
        base.Draw(renderer);

-       Shader.Bind();
+       var shader = GetAppropriateShader(renderer);
+       shader.Bind();

        // ...

-       Shader.Unbind();
+       shader.Unbind();
    }

-   protected override bool RequiresRoundedShader => ...;
+   protected override bool RequiresRoundedShader(IRenderer renderer) => ...;
}
```

`IVertex`, `TexturedVertex2D`, et al. have been re-namespaced

They are now placed under the `osu.Framework.Graphics.Rendering.Vertices` namespace

Several enums have been re-namespaced

```
WrapMode    -> osu.Framework.Graphics.Textures.WrapMode
Opacity     -> osu.Framework.Graphics.Textures.Opacity
ClearInfo   -> osu.Framework.Graphics.Rendering.ClearInfo
MaskingInfo -> osu.Framework.Graphics.Rendering.MaskingInfo
DepthInfo   -> osu.Framework.Graphics.Rendering.DepthInfo
```

2022.624.0

`TextureStore.AddStore/RemoveStore` have been split into "store" and "texture source" methods

In an effort to make the texture API easier to comprehend, the `AddStore`/`RemoveStore` methods have been split into two pairs:

- `AddTextureSource`/`RemoveTextureSource`, for adding/removing texture data lookup sources (i.e. `TextureLoaderStore`s)
- `AddStore`/`RemoveStore`, for adding/removing `TextureStore`s.

Any existing usages of `AddStore` for adding lookup sources must be changed to use `AddTextureSource` instead.

2022.607.0

"Unlimited" frame limiter is no longer completely unlimited #5235

Games using osu!framework can generally run at *very* high frame rates when not much is going on.

This can be counter-productive due to the induced allocation and GPU overhead.

- Allocation overhead can lead to excess garbage collection
- GPU overhead can lead to unexpected pipeline blocking (and stutters as a result). Also, in general graphics card manufacturers do not test their hardware at insane frame rates and therefore drivers are not optimised to handle this kind of throughput.
- We only harvest input at 1000hz, so running any higher has zero benefits.

If you think you know better for your specific application (or more correctly need to remove the limit for benchmarking), set `GameHost.AllowBenchmarkUnlimitedFrames` to `true`.

2022.528.0

`IHasFilterTerms.FilterTerms` is now an array of `LocalisableString`s

Supports filtering by either the original text or the localised form according to the currently selected language. Migration:

```
- public IEnumerable<string> FilterTerms => new string[] { ... };
+ public IEnumerable<LocalisableString> FilterTerms => new LocalisableString[] { ... };
```

`ScrollEvent.ScrollDelta.X` (horizontal scrolling) is now consistent across platforms.

Previously, Windows, Linux and Android were inverted. Delta is positive when mouse wheel scrolled to the up or left, in non-"natural" scroll mode (ie. the classic way).

2022.421.0

`IScreen` navigation interface methods now receive an "event args" structure

To allow adding more data to `IScreen` navigation interface methods in the future without further API breakage, their signatures have been changed to include an "event args" structure in the following manner:

```
- void OnEntering(IScreen last);
+ void OnEntering(ScreenTransitionEvent e);

- void OnExiting(IScreen next);
+ void OnExiting(ScreenExitEvent e);

- void OnResuming(IScreen last);
+ void OnResuming(ScreenTransitionEvent e);

- void OnSuspending(IScreen next);
+ void OnSuspending(ScreenTransitionEvent e);
```

The `last` and `next` arguments from the old signatures can now be accessed via `e.Last` and `e.Next` respectively.

Additionally, `ScreenExitEvent` contains a new `Destination` member, that allows to specify which screen is the "destination" screen of an exit operation spanning multiple screens.

2022.223.0

Visual test projects now use native .NET 6 "Hot reload"

Over the years we have maintained our own version of hot reload, affectionately named "dynamic compilation". Even after multiple complete rewrites of the system, there are still edge cases where it will unexpectedly fall over due to being too greedy in including what it considers required for the recompile.

The startup cost when running in debug was considerable (around 10-30s initialisation) and the first-compile overhead could also be high (5-60s). In addition, the dependencies required to make it work increased the final assembly size of osu!framework, even for release deploys.

With the introduction of cross-platform support for hot reload in .NET 6 we have made the decision to switch to the natively supported version.

To use this new version:

- From Rider, you'll get a popup that you have to click "Apply Changes" when a change is made to code while running. You can bind a key to `Apply Hot Reload Changes` in Rider (defaults to Alt+F10) to make it quicker to apply changes.
- Running `dotnet watch` from CLI on your test project will automatically watch files and recompile when a hange is seen.

New limitations:

- The limitations of hot reload are listed here.
- Notably, adding new `override` methods or `class`es are not supported. You can workaround this for the common scenario of prototyping new components by creating subclasses and adding the `override` methods before the initial hot reload.

We are interested in hearing feedback on this change, especially troubling cases where the previous behaviour worked better for you. Hope is that the limitations of the new method are outweighed by the leaner assembly, better performance, and (in general) up-front error when a change can't be applied, rather than an error that can be delayed longer than it would take to run a full recompile/restart.

2022.214.0

`InputManager.ChangeFocus()` will no longer switch focus to drawables that are not alive, not present or do not have a parent

To avoid unusual scenarios concerning `ChangeFocus()`, wherein a drawable could potentially request focus and have focus automatically taken away from it every frame, `ChangeFocus()` now checks whether the target drawable is alive, present and has a parent before switching focus to it.

This potentially breaks scenarios such as calling `ChangeFocus()` in `LoadComplete()` on a child drawable with the expectation that the child drawable should receive focus as soon as its ancestor is added to the draw hierarchy. In such scenarios, the suggested fix is to schedule the `ChangeFocus()` operation after children so that it is performed only when the child is fully prepared to receive focus.

In general it is recommended to check the return value of `ChangeFocus()` to determine as to whether focus was actually changed.

`CompositeDrawable.BorderColour` has changed type from `SRGBColour` to `ColourInfo`

To facilitate gradiented border support, the type of `CompositeDrawable.BorderColour` has changed from `SRGBColour` to `ColourInfo`. While some implicit conversions from `SRGBColour` to `ColourInfo` exist, some properties of `SRGBColour` are not available on `ColourInfo`, as the latter does not always represent a single colour, and may require appropriate adjustments.

2021.1118.0

`KeyBindingContainer` now sends key repeat events by default

`KeyBindingContainers` are commonly used in UI, where we expect to be able to receive key repeats. Historically this was controlled by the `SendRepeats` flag, but now that all events send `KeyBindingPressEvent` with a repeat argument, handling this legacy mode of operation was causing more issues that it was worth. The default expectation should be that repeats arrive.

For cases where key repeat may not be wanted (ie. gameplay, where your code is in complete control of user input and doesn't want to receive arbitrary key repeat events), you may opt out of receiving them by creating a subclass of `KeyBindingContainer`:

```
public class NoRepeatKeyBindingContainer : KeyBindingContainer<T>
{
    protected override bool HandleRepeats => false;
}
```

Alternatively, if you have only certain cases which you wish to opt out of, you can early return in your `OnPressed` implementation:

```
public bool OnPressed(KeyBindingPressEvent<MyAction> e)
{
    if (e.Repeat)
        return false;

    if (e.Action == MyAction.Action)
    {
       ...
    }
}
```

2021.1106.0

`BufferedContainer.CacheDrawnFrameBuffer` has been moved to a constructor argument

```
// old code:
var bufferedContainer = new BufferedContainer() { CacheDrawnFrameBuffer = true };

// new code:
var bufferedContainer = new BufferedContainer(cachedFrameBuffer: true);
```

`IEffect.CacheDrawnEffect` has been removed

We had no usages of this. If you were using it, nest the effected content in a `BufferedContainer` with `cachedFrameBuffer` set to `true`.

2021.1029.0

`TextFlowContainer` no longer returns raw `SpriteText`s, returning `ITextPart`s instead

In preparation for adding localisation support to `TextFlowContainer`s, the various `AddText()`/`AddLine()` overloads will no longer return raw `SpriteText`s. Instead, an `ITextPart` structure will be returned.

Via `ITextPart`, the consumer can both access all `Drawables` associated with a given piece of text, as well as react to any future changes in representation of the text by subscribing to `DrawablePartsRecreated`. In the future, with more localisation changes, this event will be invoked once a `LocalisableString`'s displayable content changes, which will trigger a recreation of all parts' drawables, upon which any manual adjustments applied to `Drawables` can be re-applied again.

For consumers wanting to implement their own `ITextPart`s to extend the functionality of `TextFlowContainer`, an abstract `TextPart` is also provided which implements the typical flow of handling `Drawables` and `DrawablePartsRecreated`. The only thing that a consumer has to do when inheriting that class is to implement `CreateDrawablesFor(TextFlowContainer)`.

2021.916.1

`IKeyBindingHandler<T>` and `IScrollBindingHandler<T>` now provide `UIEvent`s

To allow for more arguments without changing the signature of the handling methods, and also for consistency with the input flow in general, both interfaces now provide `UIEvent`s rather than placing each parameter directly on the methods.

```
- public bool OnPressed(T action) { }
- public bool OnScroll(T action, float amount, bool isPrecise) { }
- public void OnReleased(T action) { }
+ public bool OnPressed(KeyBindingPressEvent<T> e) { }
+ public bool OnScroll(KeyBindingScrollEvent<T> e) { }
+ public void OnReleased(KeyBindingReleaseEvent<T> e) { }
```

The following regex replacements can be used to simplify migration:

```
Find: bool OnPressed\((\w+)\s+(\w+)\)
Replace: bool OnPressed(KeyBindingPressEvent<$1> e)

Find: void OnReleased\((\w+)\s+(\w+)\)
Replace: void OnReleased(KeyBindingReleaseEvent<$1> e)

Find: bool OnScroll\((\w+) (\w+), float \w+, bool \w+\)
Replace: bool OnScroll(KeyBindingScrollEvent<$1> e)
```

2021.907.0

`AnimationClockComposite.PlaybackPosition` can no longer go below 0

This is not a change to actual playback behaviour - it only affects the return value of the playback position, which now matches the underlying playback behaviour.

2021.830.0

`ITooltip.SetContent` no longer require returning `bool` value

Until now, the method of reusing tooltip instances was problematic (two tooltips handling the same data type could not exist). Instance sharing is now based on the constructed tooltip's `Type`, rather than the data type.

An example of how you should update your code follows.

Before:

```
public override bool SetContent(object content)
{
    if (!(content is CustomContent custom))
        return false;

    text.Text = content.ToString(); // whatever you need to do here.
    return true;
}
```

After:

```
public override void SetContent(object content)
{
    text.Text = content.ToString(); // whatever you need to do here.
}
```

2021.818.0

All custom implementations of `IBindable` must implement `CreateInstance()`

Bindables previously used `Activator.CreateInstance()` to implement the `GetBoundCopy()` call. In profiling this has turned out to be a bottleneck in some scenarios, so in order to reduce the associated runtime overhead to about half, all implementors of `IBindable` now must implement `CreateInstance()`.

This also applies to all inheritors of framework-provided bindable types, who must instead override `CreateInstance()` from their base types.

For a given leaf bindable type in the inheritance hierarchy, the method should return a new constructed instance of the leaf type. The recommended pattern to use is as follows:

```
public class BaseBindable : IBindable
{
    IBindable IBindable.CreateInstance() => CreateInstance();
    protected virtual BaseBindable CreateInstance() => new BaseBindable();

    IBindable IBindable.GetBoundCopy() => GetBoundCopy();
    public BaseBindable GetBoundCopy()
        => IBindable.GetBoundCopyImplementation(this); // automatically
calls CreateInstance()
}
```

```
public sealed class CustomBindable : BaseBindable
{
    protected override BaseBindable CreateInstance() => new CustomBindable();
}
```

2021.803.0

`EnumLocalisationMapper` for enum localisation has been replaced with per-member `LocalisableDescription` attributes

As the current way for localising `enum`s require a side-class for the mapping, it became a tedious procedure to localise enums and lengthens a lot of files for supporting such.

Therefore a new `LocalisableDescription` attribute has been added allowing for localisation in a more performant and single-lined way.

```
-    [LocalisableEnum(typeof(SearchEnumLocalisationMapper))]
     public enum Search
     {
+        [LocalisableDescription(typeof(Strings), nameof(Strings.SearchHide))]
         Hide,
-        Show
-    }
-
-    public class SearchEnumLocalisationMapper : EnumLocalisationMapper<Search>
-    {
-        public override LocalisableString Map(Search value)
-        {
-            switch (value)
-            {
-                case Search.Hide:
-                    return Strings.SearchHide;
-
-                case Search.Show:
-                    return Strings.SearchShow;

-                default:
-                    throw new ArgumentOutOfRangeException(nameof(value), value, null);
-            }
-        }
+        [LocalisableDescription(typeof(Strings), nameof(Strings.SearchShow))]
```

```
+           Show
        }
```

This however require from all consumers to store the `LocalisableString`s in static providing classes, similar to [osu!'s `Strings` classes](#).

Benchmarks:

- `EnumLocalisationMapper`: | Method | Times | Mean | Error | StdDev | Median | |------------------------|------ |-------------:|-----------:|-----------:|-------------:| | GetLocalisableDescription | 1 | 6.866 us | 0.5021 us | 1.383 us | 6.316 us | | GetLocalisableDescription | 10 | 93.360 us | 4.9223 us | 13.883 us | 92.744 us | | GetLocalisableDescription | 100 | 817.775 us | 31.0268 us | 90.996 us | 812.132 us | | GetLocalisableDescription | 1000 | 10,794.319 us | 820.1783 us | 2,392.498 us | 10,718.182 us |
- `LocalisableDescription`: | Method | Times | Mean | Error | StdDev | Median | |------------------------|------ |-------------:|-----------:|-----------:|-------------:| | GetLocalisableDescription | 1 | 4.631 us | 0.1148 us | 0.3331 us | 4.514 us | | GetLocalisableDescription | 10 | 54.555 us | 1.3511 us | 3.9412 us | 54.440 us | | GetLocalisableDescription | 100 | 540.643 us | 15.9231 us | 45.6864 us | 539.138 us | | GetLocalisableDescription | 1000 | 7,218.708 us | 270.7659 us | 768.1179 us | 7,192.596 us |

[2021.721.0](#)

`PlatformAction` type is changed to an enum type

If `PlatformAction.ActionType` was matched, change to a matching of `PlatformAction` itself like:

```
bool OnPressed(PlatformAction action) {
-    switch (action.ActionType)
+    switch (action)
    {
-        case PlatformActionType.Cut:
+        case PlatformAction.Cut:
```

For `PlatformActionMethod.Delete`, and if only the Delete key should be handled (not Backspace etc.), change to `PlatformAction.Delete`:

```
-    switch (action.ActionMethod)
+    switch (action)
    {
-        case PlatformActionMethod.Delete:
+        case PlatformAction.Delete:
```

2021.628.0

`IHasTooltip.Text` is now a `LocalisableString`

`IHasTooltip.Text` now takes a `LocalisableString` instead of regular string. Users of custom tooltip containers may also have to change checks in the `SetContent()` method of their tooltip type to check if content is a `LocalisableString` instead of a regular string.

Recently added `GameThread.ThreadPausing` has been removed

This was only added in the previous release, but has since been replaced with the bindable `GameThread.State`, which gives more detail about the current state of the thread.

2021.622.0

`MarkdownHeading.GetFontSizeByLevel` now specifies absolute sizes

The result of this method is now more correctly applied to headers via `FontSize` rather than `Scale`. If you were overriding this method, please multiply your returned values by `20` to maintain sizing compatibility.

"OpenSans" is no longer provided, with the default font now being "Roboto"

We were already using Roboto in the visual tests contexts, but fallback to OpenSans could be seen for tool windows in both framework and consumer projects. In an effort to consolidate this visually, all framework components now use Roboto, the chosen font for osu!framework design logic.

This means that if you were previously relying on the presence of OpenSans and do not want to switch to Roboto, you will need to re-add the font resources in your project. Instructions on doing this can be found here.

2021.419.0

`RotationDirection.CounterClockwise` has been renamed to `Counterclockwise`

As it appears to be often one word, the enum member `CounterClockwise` has been renamed to `Counterclockwise`.

2021.416.0

`IBindable`, `IBindable<T>`, and `IBindableList<>` are no longer `IParseable`

`IParseable` is only implemented on the `Bindable<>` and `BindableList<>` classes. Code that relied on this functionality should instead cast to `IParseable`.

2021.330.0

InputHandler no longer has a Priority property

Priority is now decided by the construction order of `InputHandler`s in `CreateAvailableInputHandlers`.

2021.317.0

ConfigManager.Set is now protected and renamed to `SetDefault`

Previously this method was `public` for convenience, but as it implicitly set the `Default` value of bindables it touched, could cause unexpected behaviour.

- For initialising defaults, switch to using `SetDefault` from `InitialiseDefaults`
- For changing configuration values externally, use `SetValue` instead of `Set`

```
// previously
config.Set(ConfigType.Name, true);

// now
config.SetValue(ConfigType.Name, true);
```

osuTK support has been removed in most places

If you were supporting it via `GameHost` initialisation, you may need to remove a parameter. Internally, we still support it in a minimal way for Xamarin platforms.

2021.225.0

The regular weight of OpenSans has been renamed to OpenSans-Regular

Until now this font wasn't following the convention we use everywhere else. If you are referencing it directly, please update your strings to point to the new name.

LocalisedString no longer exists

For strings which have romanisable content, `RomanisableString` should be used instead.

Multiple UI Components now use `LocalisableString` in place of `string`

If you have custom implementations, you will need to update the overridden signatures.

Explicit `ToString` may be required where previous not

Getting the current value of some strings (ie. on UI Components) will now require an explicit call to `.ToString()`.

2021.106.0

Games will now throw (and crash) immediately on performing cross-thread transform operations

This may mean that as a framework consumer you need to re-think how you are performing operations between drawables. The easiest way to avoid issue is to use `Schedule` to force the target code to be on the correct thread.

2020.1009.0

`WaveformGraph.BaseColour` should be used instead of `Colour` for the default frequency colour

`Colour` now uniformly affects the entire graph as expected.

2020.901.0

`IAggregateAudioAdjustment.GetAggregate()` has been made an extension method

To avoid aggregate adjustment implementations from having to implement both `Aggregate{Volume,Balance,Frequency,Tempo}` and `GetAggregate()`, the latter has been made an extension method returning one of the aforementioned four values for the appropriate property of the adjustment.

Possible compilation failures related to `GetAggregate()` should be resolved by adding a using statement for the `osu.Framework.Audio` namespace to the files affected.

2020.819.0

`TextBox` events only trigger on user input

The events were specifically exposed to provide feedback on user input, but were triggering on programmatic manipulation of the textbox's text. See https://github.com/ppy/osu-framework/pull/3839/files.

2020.710.0

`TextBox` will no longer trigger sound effects

This is a fringe use-case, as the samples were never included with the framework, but if you happened to be providing them in your game resources, you will need to update your `TextBox` implementation to trigger them again. Check the osu!-side changes for an example of how to do this.

`CachedModelDependencyContainer` models must now only contain read-only fields

Mutating bindables of models attached to a `CachedModelDependencyContainer` can lead to crashes or otherwise unexpected behaviour, and is now disallowed.

2020.302.0

`FrameworkDebugConfig` no longer exposes `PerformanceLogging`

Performance logging is now toggled by opening "Global Statistics" overlay via Ctrl+F2. To manually toggle it, you may access `GameHost.PerformanceLogging`, but note that a change to this bindable will be overridden by toggling the statistics overlay.

Layout validation and invalidation has been reworked

To better cover edge-case scenarios where layout wasn't properly refreshed, the process of layout invalidation has changed.

In cases where `Invalidate()` affected a `Cached` member, the following adjustment is necessary:

```
public class MyDrawable : Drawable
{
-    private readonly Cached myLayoutValue = new Cached();
-
-    protected override bool OnInvalidate(Invalidation invalidation, Drawable source,
bool shallPropagate)
-    {
-        var result = base.OnInvalidate(invalidation, source, shallPropagate);
-
-        if ((invalidation & (Invalidation.DrawSize | Invalidation.Presence)) > 0) /* any
invalidation type of your choice */
-            result &= !myLayoutValue.Invalidate();
-
-        return result;
-    }

+    /* can also be a LayoutValue<T> */
+    private readonly LayoutValue myLayoutValue = new LayoutValue(Invalidation.DrawSize
| Invalidation.Presence);
+
+    public MyDrawable()
+    {
+        AddLayout(myLayoutValue);
+    }
}
```

In cases where `InvalidateFromChild()` affected a `Cached` member, the following adjustment is necessary:

```
public class MyDrawable : Drawable
{
-    private readonly Cached myLayoutValue = new Cached();
-
-    protected override void InvalidateFromChild(Invalidation invalidation, Drawable source)
-    {
-        if ((invalidation & (Invalidation.DrawSize | Invalidation.Presence)) > 0) /* any
invalidation type of your choice */
-            myLayoutValue.Invalidate();
-    }

+    /* can also be a LayoutValue<T> */
+    private readonly LayoutValue myLayoutValue = new LayoutValue(Invalidation.DrawSize |
Invalidation.Presence, InvalidationSource.Child);
+
```

```
+    public MyDrawable()
+    {
+        AddLayout(myLayoutValue);
+    }
}
```

In cases where *both* `Invalidate()` and `InvalidateFromChild()` affected the same `Cached` member, the following adjustment is required:

```
public class MyDrawable : Drawable
{
-    private readonly Cached myLayoutValue = new Cached();
-
-    protected override bool OnInvalidate(Invalidation invalidation, Drawable source,
bool shallPropagate)
-    {
-        var result = base.OnInvalidate(invalidation, source, shallPropagate);
-
-        if ((invalidation & Invalidation.DrawSize) > 0) /* any invalidation type of your
choice */
-            result &= !myLayoutValue.Invalidate();
-
-        return result;
-    }
-
-    protected override void InvalidateFromChild(Invalidation invalidation, Drawable source)
-    {
-        if ((invalidation & Invalidation.Presence) > 0) /* any invalidation type of your
choice */
-            myLayoutValue.Invalidate();
-    }

+    /* can also be a LayoutValue<T> */
+    private readonly LayoutValue myLocalLayoutValue = new
LayoutValue(Invalidation.DrawSize);
+    private readonly LayoutValue myChildLayoutValue = new
LayoutValue(Invalidation.Presence, InvalidationSource.Child);
+
+    public MyDrawable()
+    {
+        AddLayout(myLocalLayoutValue);
+        AddLayout(myChildLayoutValue);
+    }
```

```
+
+    protected override void Update()
+    {
+        base.Update();
+
+        /* wherever your re-validation was previously done */
+        if (!myChildLayoutValue.IsValid)
+        {
+            myLocalLayoutValue.Invalidate();
+            myChildLayoutValue.Validate();
+        }
+
+        if (!myLocalLayoutValue.IsValid)
+        {
+            /* your custom validation */
+            myLocalLayoutValue.Validate();
+        }
+    }
}
```

In cases where custom logic that can't be described as layout was done in `Invalidate()`, the following adjustment is necessary:

```
public class MyDrawable : Drawable
{
-    protected override bool OnInvalidate(Invalidation invalidation, Drawable source,
bool shallPropagate)
-    {
-        var result = base.OnInvalidate(invalidation, source, shallPropagate);
-
-        if ((invalidation & (Invalidation.DrawSize | Invalidation.Presence)) > 0) /* any
invalidation type of your choice */
-        {
-            performCustomAction();
-            result = true;
-        }
-
-        return result;
-    }

+    protected override bool OnInvalidate(Invalidation invalidation,
InvalidationSource source)
+    {
+        var result = base.OnInvalidate(invalidation, source);
+
```

```
+        if ((invalidation & (Invalidation.DrawSize | Invalidation.Presence)) > 0) /* any
invalidation type of your choice */
+        {
+            performCustomAction();
+            result = true;
+        }
+
+        return result;
+    }

    private void performCustomAction()
    {
        /* custom invalidation logic here */
    }
}
```

2020.218.0

`BindableList<T>.ItemsAdded` is now obsolete

The `ItemsAdded` event failed to provide enough context when items are moved around or inserted into the list.

It has been replaced by the `CollectionChanged` NotifyCollectionChangedEventHandler, which provides context such as the newly-added items and the indices at which the addition took place.

This event is triggered by the following methods with `Action = NotifyCollectionChangedAction.Add`:

```
BindableList<T>.Add(T item)
BindableList<T>.Insert(int index, T item)
BindableList<T>.AddRange(IEnumerable<T> items)
```

The following is an example migration utilising the new `CollectionChanged` event:

```
BindableList<int> list = new BindableList<int>();

- list.ItemsAdded += items =>
- {
-     foreach (var item in items)
-         Console.WriteLine($"Added: {item}");
- }

+ list.CollectionChanged += (_, args) =>
+ {
```

```
+     switch (args.Action)
+     {
+         case NotifyCollectionChangedAction.Add:
+             foreach (var item in args.NewItems.Cast<int>())
+                 Console.WriteLine($"Added: {item}");
+             break;
+     }
+ }
```

Note that `CollectionChanged` should *not* be used in conjunction with the `ItemsAdded` event.

`BindableList<T>.ItemsRemoved` is now obsolete

As above, the `ItemsRemoved` revent has also been replaced by the `CollectionChanged` NotifyCollectionChangedEventHandler.

This event is triggered by the following methods with `Action = NotifyCollectionChangedAction.Remove`:

```
BindableList<T>.Clear()
BindableList<T>.Remove(T item)
BindableList<T>.RemoveRange(int index, int count)
BindableList<T>.RemoveAt(int index)
BindableList<T>.RemoveAll(Predicate<T> match)
```

The following is an example migration utilising the new `CollectionChanged` event:

```
BindableList<int> list = new BindableList<int>();

- list.ItemsRemoved += items =>
- {
-     foreach (var item in items)
-         Console.WriteLine($"Removed: {item}");
- }

+ list.CollectionChanged += (_, args) =>
+ {
+     switch (args.Action)
+     {
+         // Handles both clear and remove events
+         case NotifyCollectionChangedAction.Remove:
+             foreach (var item in args.OldItems.Cast<int>())
+                 Console.WriteLine($"Removed: {item}");
+             break;
```

```
+     }
+ }
```

2020.122.0

InputManager.CreateButtonManagerFor was renamed to InputManager.CreateButtonEventManagerFor

Changed to match other methods and the class name it was creating.

Various input "end" events now return `void` and can no longer block propagation

Events affected:

```
Drawable.OnDoubleClick()
Drawable.OnDrag()
Drawable.OnDragEnd()
Drawable.OnMouseUp()
Drawable.OnKeyUp()
Drawable.OnJoystickRelease()
IKeyBindingHandler<T>.OnReleased()
```

A drawable may return `false` for an input "begin" event to allow the event to propagate further through the hierarchy. Previously, such a drawable could then return `true` for the input "end" event and prevent the event from propagating to the drawables which the input "begin" event was propagated to.
This could lead to incorrect implementations where drawables are left in weird states after handling input events.

By returning `void`, the input "end" events can no longer be blocked by other drawables in the hierarchy and are guaranteed to be invoked if their respective input "begin" event was previously invoked.

The following table illustrates the events for which the relationship is satisfied:

begin event	end event(s)
`OnDragStart()`	`OnDrag()`, `OnDragEnd()`
`OnMouseDown()`	`OnMouseUp()`
`OnKeyDown()`	`OnKeyUp()`

begin event	end event(s)
`OnJoystickPress()`	`OnJoystickRelease()`
`OnPressed()`	`OnReleased()`

2020.109.0

The namespace `osu.Framework.MathUtils` has been removed

All existing utility classes have been moved to the namespace `osu.Framework.Utils`.

2019.1224.0

`TextBox` and all related components are now abstract

`BasicTextBox` is provided as a drop-in replacement that comes with the framework design language.

`PasswordTextBox` has been renamed

Renamed to `BasicPasswordTextBox` and comes with the framework design language.

2019.1211.1

The value provided in the constructor for `Bindable<T>` is now used as both the initial and default value

The following lines of code are now identical.

```
var bindable1 = new Bindable<int>(10) { Default = 10 };
var bindable2 = new Bindable<int>(10);
```

2019.1210.1

PerformanceLogging was moved to DebugConfig

As seen in https://github.com/ppy/osu/issues/6795, some users are turning on performance logging and leaving it on. This is not intended, as it adds a noticeable overhead to retrieve and write the stack traces to disk.

This change allows the setting to reset each game execution, rather than be saved to a user's configuration file.

2019.1121.0

.NET Standard 2.1

osu!framework now targets .NET Standard 2.1. Consumers of the framework will need to install the .NET Core 3.0 SDK and change their projects to target `netstandard2.1`/`netcoreapp3.0`.

`WebRequest.ResponseString` and `WebRequest.ResponseData` properties were converted to methods

`GetResponseString` and `GetResponseData` methods are provided as a replacement. This change was done in order to indicate that these operations are expensive(they do memory allocations in the heap on each call).

2019.1104.0

`Button` has been made abstract

`BasicButton` is provided as a drop-in replacement that comes with the framework design language.

2019.921.0

`Quad.ConservativeArea` has been removed

It was inaccurate for certain `Quad`s. Use `Area` as a replacement.

2019.821.0

`SpriteText.UseFixedWidthForCharacter` has been removed

Provide an array of `char`s as `SpriteText.FixedWidthExcludeCharacters` instead.

`SpriteText.GetTextureForCharacter` and `SpriteText.GetFallbackTextureForCharacter` have been removed.

Glyphs must now always be retrieved through an `ITexturedGlyphLookupStore`. The `SpriteText.CreateTextBuilder()` method is provided to allow overriding the store which glyphs are retrieved from.

`BlendingModes` are now obsoleted

Please switch to using `BlendingParameters` static properties instead, whcih provide the same functionality. This change was made to expose full control over blend equations and simplify the blending class structure, which previously spanned three related types.

2019.809.0

`Cached` is now a class and requires object initialisation

Usages of `Cached` without initialising (via `new Cached()`) will need to be updated.

`Animation` no longer supports `AutoSizeAxes`

`Animation` will automatically auto-size on axes which are not relative sized (via `RelativeSizeAxes`) and for which `Size` has not been set.

2019.726.0

`Path` now supports `AutoSizeAxes` and is set to auto-size in both axes by default

There are now three modes of operation for paths:

```
// Auto size
new Path()

// Static size
new Path
{
    AutoSizeAxes = Axes.None,
    Size = new Vector2(100)
}

// Relative size
new Container
{
    Size = new Vector2(100),
    new Path
    {
        AutoSizeAxes = Axes.None,
        RelativeSizeAxes = Axes.Both
```

```
    }
}
```

2019.702.0

`InputKey` enum names for extra mouse buttons have been renamed.

```
MouseButtonX -> ExtraMouseButtonX
```

2019.628.0

`Menu` and several related components have been made abstract

For each component, there are two ways to resolve resulting errors:

1. `ContextMenuContainer`
    1. `BasicContextMenuContainer` is provided as a drop-in replacement that comes with the framework design language.
    2. Implement your own local `ContextMenuContainer`. The following is consumable code that matches the removed implementation. Rename it to suit your project and update your references:

```
public class MyContextMenuContainer : ContextMenuContainer
{
    // If you have a custom menu, provide it here.
    protected override Menu CreateMenu() => new BasicMenu(Direction.Vertical);
}
```

2. `Menu`
    1. `BasicMenu` is provided as a drop-in replacement that comes with the framework design language.
    2. Implement your own local `Menu`. The following is consumable code that matches the removed implementation. Rename it to suit your project and update your references:

```
public class MyMenu : Menu
{
    public MyMenu(Direction direction, bool topLevelMenu = false)
        : base(direction, topLevelMenu)
    {
    }
```

```
        protected override Menu CreateSubMenu() => new MyMenu(Direction.Vertical);

        protected override DrawableMenuItem CreateDrawableMenuItem(MenuItem item) =>
    new MyDrawableMenuItem(item);

        // If you have a custom scroll container, provide it here.
        protected override ScrollContainer<Drawable> CreateScrollContainer(Direction
    direction) => new BasicScrollContainer(direction);

        private class MyDrawableMenuItem : DrawableMenuItem
        {
            public MyDrawableMenuItem(MenuItem item)
                : base(item)
            {
            }

            protected override Drawable CreateContent() => new SpriteText
            {
                Anchor = Anchor.CentreLeft,
                Origin = Anchor.CentreLeft,
                Padding = new MarginPadding(5),
                Font = new FontUsage(size: 17),
            };
        }
    }
```

3. `DropdownMenu` / `DropdownMenuItem`. This is only applicable if you already implement a custom `Dropdown<T>`.
    1. The following is consumable code that matches the removed implementation. Rename it to suit your project and update your references:

```
    public class MyDropdown<T> : Dropdown<T>
    {
        // If you have a custom header, provide it here.
        protected override DropdownHeader CreateHeader() => null;

        protected override DropdownMenu CreateMenu() => new MyDropdownMenu();

        private class MyDropdownMenu : DropdownMenu
        {
            // If you have a custom menu, provide it here.
            protected override Menu CreateSubMenu() => new BasicMenu(Direction);

            // If you have a custom scroll container, provide it here.
            protected override ScrollContainer<Drawable> CreateScrollContainer(Direction
```

```
direction) => new BasicScrollContainer(direction);

        protected override DrawableDropdownMenuItem
CreateDrawableDropdownMenuItem(MenuItem item) => new
MyDrawableDropdownMenuItem(item);

        private class MyDrawableDropdownMenuItem : DrawableDropdownMenuItem
        {
            public MyDrawableDropdownMenuItem(MenuItem item)
                : base(item)
            {
            }

            // If you have custom content, provide it here.
            protected override Drawable CreateContent() => new SpriteText
            {
                Anchor = Anchor.CentreLeft,
                Origin = Anchor.CentreLeft,
                Padding = new MarginPadding(5),
                Font = new FontUsage(size: 17),
            };
        }
    }
}
```

`DebugSetting.ActiveGCMode` no longer exists

If you were manually setting this, you can do so via .NET methods.

2019.614.0

`ScrollContainer<T>` is now abstract (and `ScrollContainer` no longer exists)

In a push to make the provided framework components design agnostic, `ScrollContainer` is no longer provided. As a framework consumer you have a few options to resolve this:

- Use `BasicScrollContainer`

The framework now provides `BasicScrollContainer` and `BasicScrollContainer<T>`. These are drop-in replacements but come with the framework design language, so it is only recommended as a temporary solution.

- Implement your own local `ScrollContainer`

Here is consumable code to provide a local implementation that matches the removed one. Rename to suit your project and update your references:

```
public class MyScrollContainer : ScrollContainer<Drawable>
{
    public MyScrollContainer(Direction scrollDirection = Direction.Vertical)
        : base(scrollDirection)
    {
    }

    protected override ScrollbarContainer CreateScrollbar(Direction direction) =>
new MyScrollbar(direction);

    protected class MyScrollbar : ScrollbarContainer
    {
        private const float dim_size = 10;

        private readonly Color4 hoverColour = Color4.White;
        private readonly Color4 defaultColour = Color4.Gray;
        private readonly Color4 highlightColour = Color4.Black;

        private readonly Box box;

        public MyScrollbar(Direction scrollDir)
            : base(scrollDir)
        {
            Colour = defaultColour;

            Blending = BlendingMode.Additive;

            CornerRadius = 5;

            const float margin = 3;

            Margin = new MarginPadding
            {
                Left = scrollDir == Direction.Vertical ? margin : 0,
                Right = scrollDir == Direction.Vertical ? margin : 0,
                Top = scrollDir == Direction.Horizontal ? margin : 0,
                Bottom = scrollDir == Direction.Horizontal ? margin : 0,
            };

            Masking = true;
            Child = box = new Box { RelativeSizeAxes = Axes.Both };
```

```
            ResizeTo(1);
        }

        public override void ResizeTo(float val, int duration = 0, Easing easing
= Easing.None)
        {
            Vector2 size = new Vector2(dim_size)
            {
                [(int)ScrollDirection] = val
            };
            this.ResizeTo(size, duration, easing);
        }

        protected override bool OnHover(HoverEvent e)
        {
            this.FadeColour(hoverColour, 100);
            return true;
        }

        protected override void OnHoverLost(HoverLostEvent e)
        {
            this.FadeColour(defaultColour, 100);
        }

        protected override bool OnMouseDown(MouseDownEvent e)
        {
            if (!base.OnMouseDown(e)) return false;

            //note that we are changing the colour of the box here as to not interfere with
the hover effect.
            box.FadeColour(highlightColour, 100);
            return true;
        }

        protected override bool OnMouseUp(MouseUpEvent e)
        {
            if (e.Button != MouseButton.Left) return false;

            box.FadeColour(Color4.White, 100);

            return base.OnMouseUp(e);
        }
    }
}
```

2019.611.0

`LinearVertexBuffer` and `QuadVertexBuffer` can no longer be constructed outside of osu!framework.

For existing code, `LinearBatch(size, 1)` and `QuadBatch(size, 1)` may be used to replace vertex buffer usages. Code must be updated to always invoke `batch.Add(vertex)` with the vertices that should be drawn.

`VertexBuffer` may still be derived for custom implementations.

VisibilityContainer now exposes visibility via a bindable

`VisibilityContainer` no longer implements `IStateful<Visibility>`, instead exposing a single `Bindable<Visibility> State`.

2019.606.0

`Texture.DrawTriangle()` and `Texture.DrawQuad()` have been removed (#2475)

Use `DrawNode.DrawTriangle()` and `DrawNode.DrawQuad()` instead.

2019.604.0

Global `TrackStore` and `ResourceStore`s can no longer access resources that are out of their respective namespaces.

Previously, it was possible to access any resource as a part of `TrackStore` and `SampleStore` via their `Get` methods because the entire resource store would be nested inside them.

This nesting has been removed, so you can now only access files that are part of their specified namespaces (folders).

- For `TrackStore`, this will be the `Tracks` namespace.
- For `SampleStore`, this will be the `Samples` namespace.

Audio stores have been renamed

To bring naming in-line with the purpose of audio stores, the following classes have been renamed:

`SampleManager` -> `SampleStore`
`TrackManager` -> `TrackStore`

In line with this, return values of `GetSampleStore` and `GetTrackStore` now return `ISampleStore` and `ITrackStore` respectively, hiding the underlying manager logic.

The public variables relating to these stores have been renamed to reflect this change:

`AudioManager.Track` -> `AudioManager.Tracks`
`AudioManager.Sample` -> `AudioManager.Samples`

Virtual tracks must be retrieved from `ITrackStore`

`ITrackStore` now has a `GetVirtual` method which will create a virtual track. This will handle adding the track correctly to the audio subsystem. If one wishes to create a custom type of virtual tracks (a super-edge-case) you could implement the `ITrackStore` interface and manually add your component to `AudioManager` via `AddItem`.

2019.523.0

TabControl can now select nothing #2430

While this does not match most OS implementations of a tab control, this was deemed useful for o!f usage scenarios.

Previously, this code would throw an exception.

```
tabControl.Current.Value = null;
```

Now it is allowed.

Dropdown can now select nothing #2428

Previously, this code would throw an exception.

```
dropdownMenu.Current.Value = null;
```

Now it is allowed. Note that implementations of dropdown should be updated to handle this (common scenario is that the `DropdownHeader` would not correctly handle a `string.Empty` case if it was using AutoSize in the Y axis).

`TabItem.IsRemovable` is true by default #2425

This felt like a more sane default. Any existing usage of custom `TabItem`s where `IsRemovable` was overridden, or any existing usage where removal is explicitly *not* wanted need to be updated.

2019.514.0

`Game.FrameStatisticsMode` is now a bindable named `Game.FrameStatistics` #2399

This allows consumers to hook value changed events so they can, for instance, save the (framework) FPS display's state to config and handle hotkey-based changes.

Visual `TestCase`s are now `TestScene`s #2365

The term `TestCase` is used by testing frameworks to denote single methods inside a test class. We were using it on the class itself as a prefix, which got quite confusing. This resolves the conflict and feels more correct as these are visual tests – ie. "scenes".

Note that the `TestCase` prefix is still supported by tooling for the time being. We still recommend you update (via a simple rename) as soon as possible.

2019.427.0

`Drawable.ApplyDrawNode()` has been removed #2314

The direction of application of `DrawNode` states has been inversed.

Previously:

```
class MyCustomDrawable : Drawable
{
    private bool state;

    protected override DrawNode CreateDrawNode() => new CustomDrawNode();

    protected override void ApplyDrawNode(DrawNode node)
    {
        base.ApplyDrawNode(node);

        var n = (CustomDrawNode)n;

        n.State = state;
    }

    private class CustomDrawNode : DrawNode
```

```
    {
        public bool State;
    }
}
```

Now:

```
class MyCustomDrawable : Drawable
{
    private bool state;

    protected override DrawNode CreateDrawNode() => new CustomDrawNode(this);

    private class CustomDrawNode : DrawNode
    {
        protected new MyCustomDrawable Source => (MyCustomDrawable)base.Source;

        private bool state;

        public CustomDrawNode(MyCustomDrawable source)
            : base(source)
        {
        }

        public override void ApplyState()
        {
            base.ApplyState();

            state = Source.state;
        }
    }
}
```

The namespace of `EdgeEffectParameters` and `EdgeEffectType` has changed #2314

They now reside in `osu.Framework.Graphics.Effects`.

2019.327.0

Font fallback order changes #2296

Games adding their own fonts can now do so directly to `Game.Fonts`. Fallback (framework-side) fonts are now always at the lowest priority.

Previously:

```
// this completely overrides the framework default. will need to change once we make a
proper FontStore.
dependencies.Cache(Fonts = new FontStore(new GlyphStore(Resources, @"Fonts/FontAwesome")));
Fonts.AddStore(new GlyphStore(Resources, @"Fonts/osuFont"));
```

now:

```
Fonts.AddStore(new GlyphStore(Resources, @"Fonts/osuFont"));
```

FontAwesome and SpriteIcon is provided by the framework #2293

If you manually added either of these to your project, please remove them.

Usage:

```
new SpriteIcon
{
    Icon = FontAwesome.Refresh,
};
```

2019.319.0

Parameter order of `AddUntilStep` / `AddWaitStep` reversed #2256

In order to standardise parameter order, these two methods' parameters have been reversed. Old parameter order methods are available temporarily (as `[Obsolete]`) to ease migration.

New attribute `[SetUpSteps]` added #2266

Finally allows steps to be added that will run before every `[Test]` method.

2019.221.0

Implicit operator removed from `Bindable` #2152

In order to avoid accidental misuse / misunderstandings, `.Value` must always be added to `Bindable` when requesting its value. One such case which used to be confusing:

```
Bindable<Drawable> bindableDrawable = new Bindable<Drawable>();

if (bindableDrawable != null)
{
    // true
}

if (bindableDrawable.Value != null)
{
    // false
}

Drawable drawable = null;

if (bindableDrawable == drawable)
{
    // ???
}
```

Bindable.ValueChanged now provides `ValueChangedEvent` #2012

It is now possible to access the old value. This was a common requirement in bindable usage which we had issues with until now.

```
bindable.ValueChanged += e =>
{
    Console.WriteLine($"Value changed from {e.OldValue} to {e.NewValue}");
}
```

Introduction of `FontUsage` #2043

While old methods of setting font attributes will still work, they are now marked as `[Obsoleted]`. You should use `Font = new FontUsage(size: 200)` going forward. For convenience, you can adjust existing fonts like so:

```
var font = new FontUsage(family: "SourceCodePro");

var fontWithSize = font.With(size: 24);
```

Bindable Flow

`osu-framework` utilizes `Bindable<T>` objects to distribute data between components. In conjunction with Drawable components, they provide functionality to automatically remove communication between `Bindable<T>` objects when finalized, serving as a safer alternative to C#'s `event`.

Creating a `Bindable<T>`

In `public`/`protected`/`internal` scenarios, it is recommended to store a private `Bindable<T>` backing and re-expose it publicly as one of the read-only interfaces - `IBindable` or `IBindable<T>`, depending on how much access the outside objects should have.

In `private` scenarios, it is simplest to store bindables as `Bindable<T>`.

```
public class MyClass
{
    private readonly Bindable<int> privateBacking = new Bindable<int>();
    public IBindable<int> PublicBindable => privateBacking;

    private readonly Bindable<int> privateBindable = new Bindable<int>();
}
```

A few subclasses, such as `BindableDouble`, extend `Bindable<T>` to provide additional functionality such as range-limiting of numeric values or floating point precision handling. These are available under the `osu.Framework.Bindables` namespace.

Binding `Bindable<T>`s together

`Bindable<T>` supports the ability to "bind" to other `Bindable<T>`s. When either one of the bindable's values changes, it will also set the value on the other.

This is available through the `.BindTo()` and `.GetBoundCopy()` methods, as well as the `.BindTarget` property (the latter is especially useful as syntactic sugar in property initialiser usage).

```
var x = new Bindable<int>(1);
var y = new Bindable<int>(2);

x.BindTo(y);
Assert.That(x.Value, () => Is.EqualTo(2)); // BoundTo() immediately sets x's value to y's

y.Value = 10;
Assert.That(x.Value, () => Is.EqualTo(10)); // The value set to y above is propagated to x
```

```
x.Value = 5;
Assert.That(y.Value, () => Is.EqualTo(5)); // Same as above - dual-way communication


var x = new Bindable<int>(1);
var y = x.GetBoundCopy(); // The binding is set up automatically here.

y.Value = 10;
Assert.That(x.Value, () => Is.EqualTo(10)); // The value set to y above is propagated to x.

x.Value = 5;
Assert.That(y.Value, () => Is.EqualTo(5)); // Same as above - dual-way communication


var x = new Bindable<int>(1);
var y = new Bindable<int>() { BindTarget = x };

y.Value = 10;
Assert.That(x.Value, () => Is.EqualTo(10)); // The value set to y above is propagated to x.

x.Value = 5;
Assert.That(y.Value, () => Is.EqualTo(5)); // Same as above - dual-way communication
```

Generally, when interacting with a bindable exposed by another object, one should always make a local bindable to track changes. **Binding to another object instance's bindable events should be avoided**, as it bypasses the reference safety provided by bindables.

Observing the values of a `Bindable<T>`

The value of a `Bindable<T>` can be observed through the `ValueChanged` event. This returns the new value.

```
var x = new Bindable<int>();

x.ValueChanged += val => Assert.IsTrue(val.NewValue == 2);
x.Value = 2;
```

In some scenarios it may be desirable to have the `ValueChanged` delegate invoked once after being set up. It is possible to achieve this in two ways:

```
var x = new Bindable<int>();

// #1:
x.BindValueChanged(myValueFunc, true); // Recommended
```

```
// #2:
x.ValueChanged += myValueFunc;
x.TriggerChange();
```

Breaking the `Bindable<T>` chain

The binding chain between `Bindable<T>`s is weak. That is, bound `Bindable<T>`s will not cause each other to forego garbage collection.

When a `Bindable<T>` is finalized, it will automatically unbind other `Bindable<T>`s bound via `BindTo()` and any subscribers to its `ValueChanged` event. This is subject to the garbage collector and may not occur instantly.

When a `Drawable` disposes, it forces all `Bindable<T>`s contained as `private`/`protected`/`public`/`internal` fields and properties to unbind. This generally occurs as soon as the `Drawable` is removed from the draw hierarchy via `Clear(true)`/`ClearInternal(true)` or setting `InternalChildren`/`Children`, or via the garbage collector when all references to the `Drawable` are lost.

It may be desired to unbind at an arbitrary point in time when finalization is not guaranteed by the above. A few methods are provided to achieve this:

- `UnbindEvents()` - Unbinds all subscribers bound via `ValueChanged`.
- `UnbindBindings()` - Unbinds all `Bindable<T>`s bound via `BoundTo()`.
- `UnbindAll()` - Combines `UnbindEvents()` and `UnbindBindings()`.

Leasing

There may be a scenario where you want to restrict updates to a high level bindable, but still allow changes by a certain component (or subset of components). Leasing exists to fulfil this goal. While a bindable is leased, it will be set to a `Disabled` state, but the special `LeasedBindable` will bypass this check and allow the bindable's value to be changed (and propagate as usual).

```
var x = new Bindable<int>();
var leased = x.BeginLease(false);

// x.Value == 0
// leased.Value == 0

// x.Disabled == true
// leased.Disabled == true

x.Value = 1; // invalid, will throw an exception
leased.Value = 1; // valid, even though disabled
```

```
// x.Value == 1
// leased.Value == 1

leased.Return();
```

Another use case for leasing is to take advantage of the reverting of value on `Return()`. This allows a subsystem to gain control over a bindable, make changes, and after that subsystem is done, return the original bindable to its original state.

```
var x = new Bindable<int>(2);
var leased = x.BeginLease(true);

leased.Value = 1;

// x.Value == 1
leased.Return();
// x.Value == 2
```

Note that during a lease, the `Disabled` state may be changed from the `LeasedBindable`. It will be restored to the value before the lease began on `Return()`.

Input handling follows a topological model whereby the top-most (visually) `Drawable` handles input prior to anything else in the game. Hierarchically between a single parent-child relationship, the child handles input before the parent.

```
Parent              < Handler #4
    Child_1         < Handler #3
    Child_2         < Handler #2
        Child_2_1   < Handler #1
```

Positional vs non-positional input

"Positional" input refers to any input that depends on a screen-space position (e.g. hover). "Non-positional" input refers to any other input (e.g. a button press).

Individual event handlers

Positional

```
protected virtual bool OnMouseMove(MouseMoveEvent e);

protected virtual bool OnHover(HoverEvent e);
protected virtual void OnHoverLost(HoverLostEvent e);

protected virtual bool OnMouseDown(MouseDownEvent e);
protected virtual void OnMouseUp(MouseUpEvent e);

protected virtual bool OnClick(ClickEvent e);
protected virtual bool OnDoubleClick(DoubleClickEvent e);

protected virtual bool OnDragStart(DragStartEvent e);
protected virtual void OnDrag(DragEvent e);
protected virtual void OnDragEnd(DragEndEvent e);

protected virtual bool OnScroll(ScrollEvent e);

protected virtual bool OnTouchDown(TouchDownEvent e);
protected virtual void OnTouchMove(TouchMoveEvent e);
protected virtual void OnTouchUp(TouchUpEvent e);

protected virtual bool OnTabletPenButtonPress(TabletPenButtonPressEvent e);
protected virtual void OnTabletPenButtonRelease(TabletPenButtonReleaseEvent e);
```

Non-positional

```
protected virtual bool OnKeyDown(KeyDownEvent e);
protected virtual void OnKeyUp(KeyUpEvent e);

protected virtual bool OnJoystickPress(JoystickPressEvent e);
protected virtual void OnJoystickRelease(JoystickReleaseEvent e);
protected virtual bool OnJoystickAxisMove(JoystickAxisMoveEvent e);

protected virtual bool OnMidiDown(MidiDownEvent e);
protected virtual void OnMidiUp(MidiUpEvent e);

protected virtual bool OnTabletAuxiliaryButtonPress(TabletAuxiliaryButtonPressEvent e);
protected virtual void OnTabletAuxiliaryButtonRelease(TabletAuxiliaryButtonReleaseEvent e);
```

These event handlers should be used when singular events are to be handled.

The boolean return value indicates whether the event should continue being propagated to other `Drawable`s in the scene graph.

> **Example:** If a child did not want its parent to receive an `OnHover()` event, it should override `OnHover()` to return `true`.

`OnHoverLost()` is an exception which is unconditionally invoked on every un-hovered `Drawable`.

Example:

```
protected override bool OnClick(ClickEvent e)
{
    this.ScaleTo(1.25f, 50).Then().ScaleTo(1f, 50);
    return true;
}
```

Input handlers that correspond to continuations/resolutions of previous input, such as `OnMouseUp()`, `OnDragEnd()`, `OnKeyUp()`, etc., will only fire on drawables that registered to handle the original input by returning `true` - for the examples above, that would be `OnMouseDown()`, `OnDragStart()`, `OnKeyDown()`, etc. Those events cannot be suppressed.

Aggregate event handler

```
protected virtual bool Handle(UIEvent e);
```

This event handler should be used when groups of events with identical implementations are to be handled.

This counts as both a positional and non-positional event handler and works well when combined with the `HandleNonPositionalInput` and `HandlePositionalInput` properties described below.

> **Example**
>
> ```
> protected override bool Handle(UIEvent e)
> {
>     switch (e)
>     {
>         case MouseEvent _:
>             // Stop all mouse events from being handled by other Drawables
>             return true;
>
>         default:
>             // Let other Drawables handle everything else
>             return false;
>     }
> }
> ```

Controlling whether input is to be handled

By default, any `Drawable` that implements the handlers described above will receive all events appropriate for the types of input listed.

```
public virtual bool HandleNonPositionalInput;
public virtual bool HandlePositionalInput;
```

The above properties are provided to control whether a `Drawable` should receive the types of input events.

> **Example:** If a `Drawable` did not want to handle any click, drag, and hover events at some point, it could achieve this by overriding `HandlePositionalInput` to return `false`.

> **Warning:** Overriding the above properties will cause the `Drawable` to be *considered* for (but not necessarily receive) that type of input, and thus serves as an anti-optimisation if the intention is to make the `Drawable` not considered for that type of input.

If a parent should control whether itself or its children should be considered for a type of input at all, the following properties are provided to control the behaviour:

```
public virtual bool PropagateNonPositionalInputSubTree;
public virtual bool PropagatePositionalInputSubTree;
```

Example:

```
class MyContainer : FillFlowContainer
{
    public MyContainer()
    {
        for (int i = 0; i < 1024; i++)
            Add(new Button { Size = new Vector2(50) });
    }

    // Whoah! The buttons we added are just for show, they don't actually handle clicks!
    public override bool PropagatePositionalInputSubTree => false;
}
```

Receiving and handling focus

```
public virtual bool RequestsFocus;
public virtual bool AcceptsFocus;

protected virtual void OnFocus(FocusEvent e);
protected virtual void OnFocusLost(FocusLostEvent e);
```

Focus may be received via both positional and non-positional input. A `Drawable` with `AcceptsFocus = false` will never receive focus.

Positional

A `Drawable` receives focus when `OnClick()` returns `true`.

Non-positional

The top-most `Drawable` with `RequestsFocus = true` and `HandleNonPostionalInput = true` will receive focus if nothing else has focus.

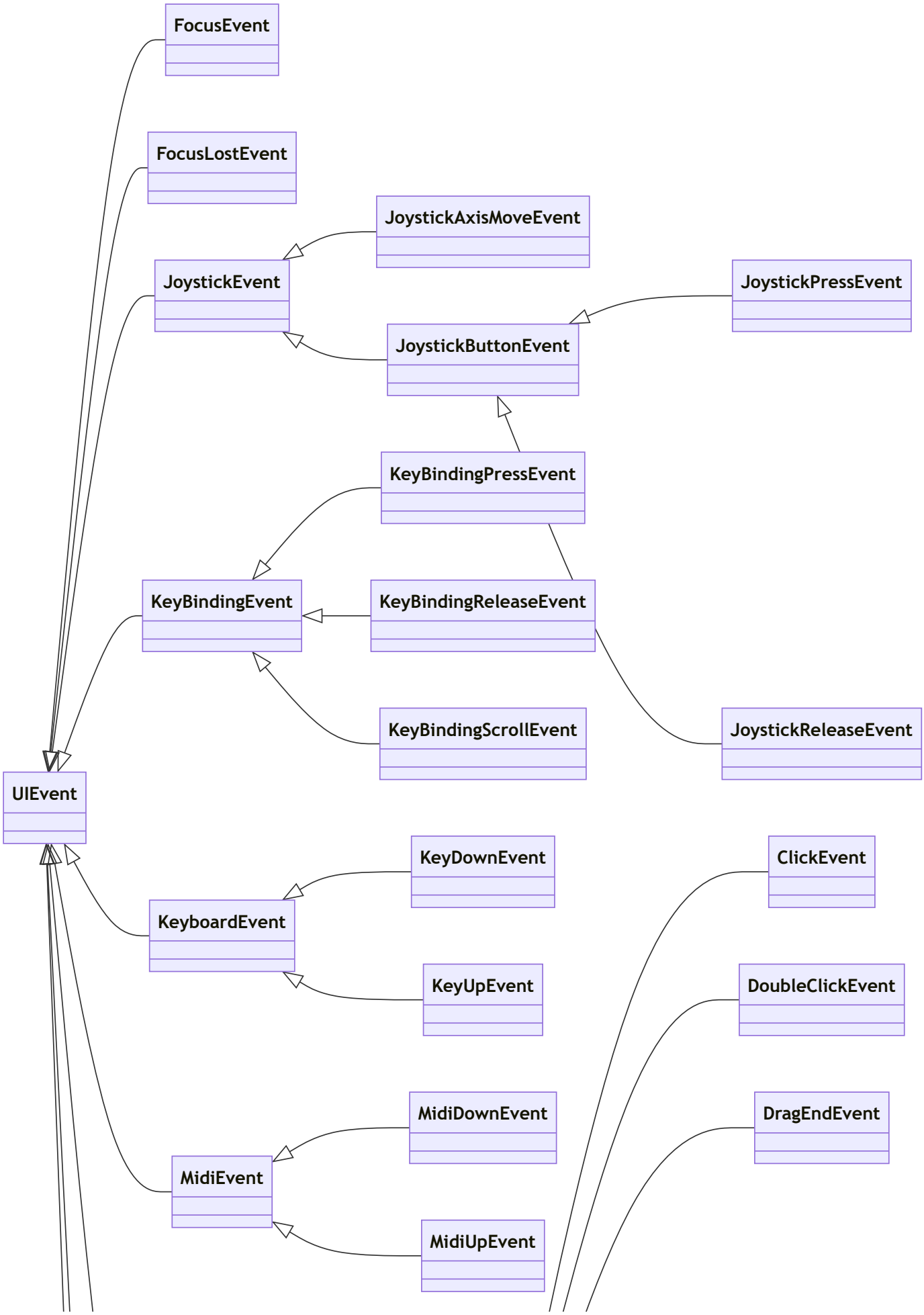
The `OnFocusLost()` method is unconditionally invoked on the un-focused `Drawable`.

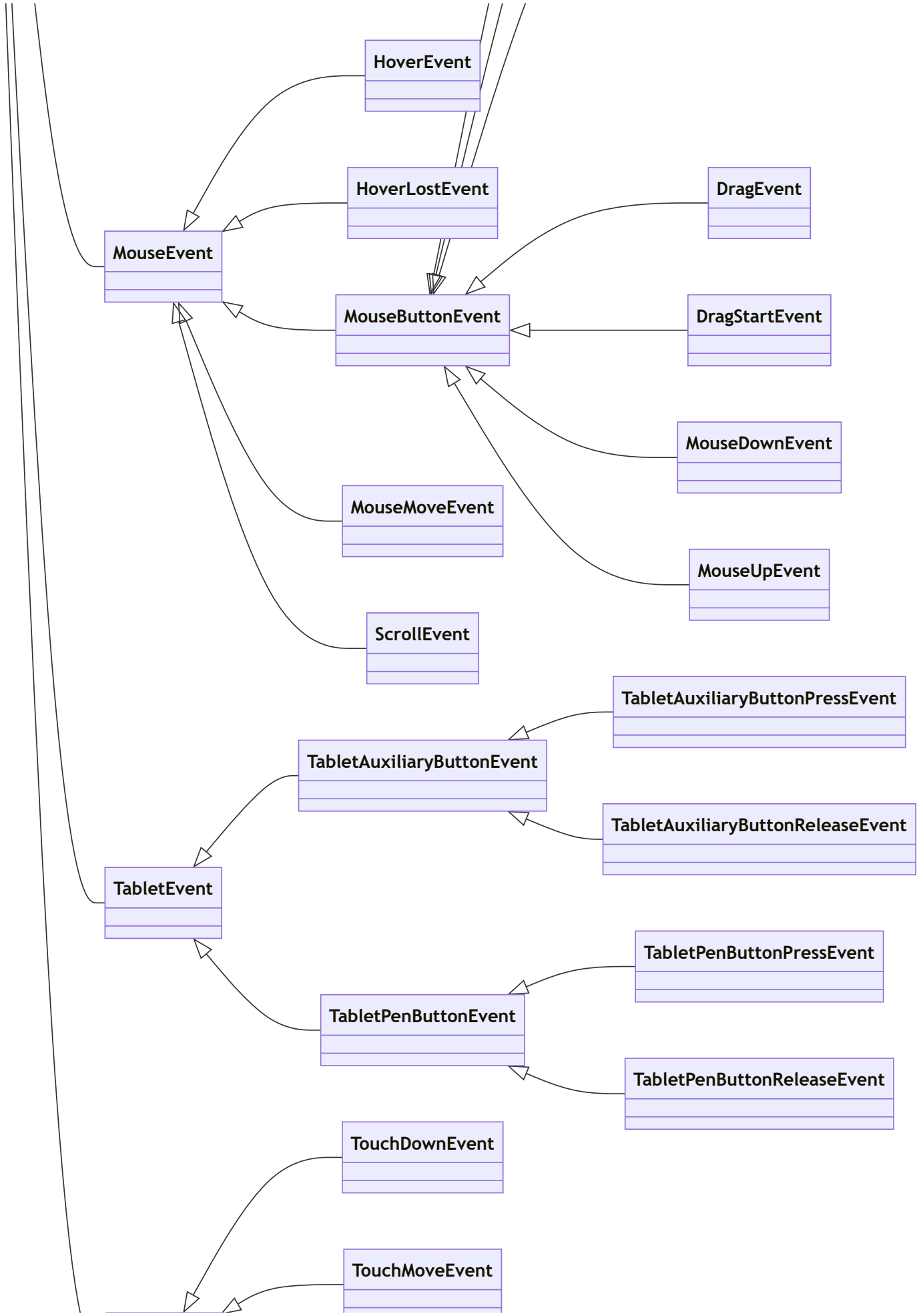
> **Example:**

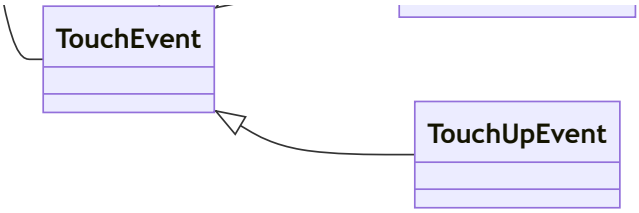
```
class MyDrawable : CompositeDrawable
{
    public override bool AcceptsFocus => true;
    protected override bool OnClick(ClickEvent e) => true;

    protected override void OnFocus(FocusEvent e) => this.ScaleTo(1.5f, 50);
    protected override void OnFocusLost(FocusLostEvent e) => this.ScaleTo(1f, 50);
}
```

Input event hierarchy

The following hierarchical relationship of input events can be useful to know when drilling down with the aggregate event handler.


FocusEvent
FocusLostEvent
JoystickAxisMoveEvent
JoystickEvent
JoystickPressEvent
JoystickButtonEvent
KeyBindingPressEvent
KeyBindingEvent
KeyBindingReleaseEvent
KeyBindingScrollEvent
JoystickReleaseEvent
UIEvent
KeyDownEvent
ClickEvent
KeyboardEvent
KeyUpEvent
DoubleClickEvent
MidiDownEvent
DragEndEvent
MidiEvent
MidiUpEvent


HoverEvent
HoverLostEvent
DragEvent
MouseEvent
MouseButtonEvent
DragStartEvent
MouseDownEvent
MouseMoveEvent
MouseUpEvent
ScrollEvent
TabletAuxiliaryButtonPressEvent
TabletAuxiliaryButtonEvent
TabletAuxiliaryButtonReleaseEvent
TabletEvent
TabletPenButtonPressEvent
TabletPenButtonEvent
TabletPenButtonReleaseEvent
TouchDownEvent
TouchMoveEvent


TouchEvent
TouchUpEvent

Asynchronous loading

osu!framework puts a lot of focus on non-blocking loading, striving to never block the interface threads at any point. It is therefore recommended (and in some cases forced) that you use async patterns when loading components.

Preloading components

It is recommended that any larger components are preloaded ahead of time. For instance, after a user has arrived on a menu screen, loading could begin on the next screen(s) that will be displayed.

The simplest method to load a component in the background is by calling `LoadComponentAsync()` from inside a `Drawable`. This method also exposes a callback which can be used to perform an action when loading completes, which can be useful for showing a newly loading component as soon as it is ready.

Instantly using the component:

```
LoadComponentAsync(new MyComponent(), Add);
```

Preloading for future use:

```
var myComponent = new MyComponent();

LoadComponentAsync(myComponent);

// ...

Add(myComponent); // note that this will cause a blocking load if the async load has not
yet completed.
```

This method returns a `Task` which can be blocked on with `Task.Wait()` if you need custom blocking behaviour.

Asynchronous chaining with `[BackgroundDependencyLoader]`

When a component is loaded, it will attempt to load all nested children that have `ShouldBeAlive == true`. To make use of this chaining, ensure that all loading code is either in the `ctor` or a private method marked with the `[BackgroundDependencyLoader]` attribute. The latter is recommended as it avoids a potential overhead when constructing new instances of the component that aren't nested in an asynchronous load.

```
public class MainComponent : CompositeDrawable
{
    [BackgroundDependencyLoader]
    private void load()
    {
        Child = new NestedComponent();
        // this could also be in the ctor if required, but generally use BDL wherever
possible for maximum efficiency.
    }
}

public class NestedComponent : Drawable
{
    [BackgroundDependencyLoader]
    private void load()
    {
        // long running load
    }
}
```

The following call could then be used to load `MainComponent` and `NestedComponent` asynchronously, only adding MainCompoent when the nested tree is completely loaded.

```
LoadComponentAsync(new MainComponent(), Add);
```

Overriding pre-packaged asynchronous behaviour

Some classes come with pre-built asynchronous behaviours. One example is `Screen`, which automatically loads a child screen that is `Push`ed if it is not already in a loaded state. Overriding such asynchronous behaviour can be achieved as mentioned above, by manually blocking on a preload call:

```
var blocking = new BlockingScreen();

LoadComponentAsync(blocking).Wait();
screenStack.Push(blocking);
```

Long-running load components (online retrieval etc.)

The special attribute `[LongRunningLoad]` exists to mark components which require loads that take a long time (generally anything >100ms). This is usually a drawable which is retrieving a texture (or other content) from a network source. By attaching this to a class, you can ensure that all usage of that class is correctly loaded in an asynchronous context.

There are two important things to note about this attribute:

Marked components will run in their own segregated thread pool

This avoids any potential of thread pool saturation by network requests or otherwise. Normal components loaded via `LoadComponentAsync()` will be unaffected by `LongRunningLoad` components.

Marked components must be a top-level async load

In order to avoid `[LongRunningLoad]` components accidentally ending up on the standard async load thread pool, they must always be run *directly* via `LoadComponentAsync()`. An exception will be thrown if this is not the case.

Dependency Injection

In osu!framework, we support dependency injection at a `Drawable` level. Internally, this is done via `DependencyContainer`s, which are passed down the hierarchy and can be overridden at any point for further customisation (or replacement) by a child.

The general usage for this is to fulfill a dependency that can come from a parent (potentially many levels above the point of usage). It is important to understand the general concept of dependency injection before reading on.

Implementation

osu!framework's dependency injection mechanism heavily leans on C# attributes, namely `[Cached]`, `[Resolved]`, and `[BackgroundDependencyLoader]`. Setting the dependencies up is done via one of two pathways: source generation and reflection. Understanding this is key, as the source generation pathway benefits from compile-time optimisations, but requires consumers to adjust their code accordingly.

Source generation

Since the 2022.1126.0 release, the primary supported implementation of dependency injection relies on source generators. The primary implications of this for framework consumers are as follows:

- For the source generator-based dependency injection to work, `Drawable` classes must be `partial` so that the source generator can inject the DI machinery into the class. Non-compliant drawables will raise the `OFSG001` code inspection.
- In more complicated custom DI usages, if it is desired to `.Inject()` dependencies into a custom non-drawable class, it must implement the marker `IDependencyInjectionCandidate` interface.

The implementation of the source generator can be viewed here.

For a fast compile-run cycle, source generators are by default only ran on release builds. Debug builds will use the reflection pathway as a fallback.

Reflection

The original, legacy implementation of dependency injection heavily uses reflection. It will be used if user drawables are not marked `partial`, as the source generator cannot attach its own code to such drawables.

Since the source generator pathway was introduced, this implementation is supported for backwards compatibility, but generally not recommended for new projects.

Storing and retrieving dependencies

There are a few ways dependencies can be **cached** (stored) and **resolved** (retrieved):

[Cached] on drawable members

This is the simplest implementation.

```
/// <summary>
/// A class which caches something for use by children.
/// </summary>
public partial class MyGame : Game
{
    [Cached]
    protected readonly MyStore Store = new MyStore();

    public MyGame()
    {
        Child = new Container
        {
            RelativeSizeAxes = Axes.Both,
            Child = new Container
            {
                RelativeSizeAxes = Axes.Both,
                Child = new Container
                {
                    RelativeSizeAxes = Axes.Both,
                    Child = new MyComponent()
                }
            },
        };
    }
}

/// <summary>
/// A component that consumed the cached class.
/// </summary>
public partial class MyComponent : CompositeDrawable
{
    [Resolved]
    protected MyStore FetchedStore { get; private set; } = null!;

    protected override void LoadComplete()
    {
        base.LoadComplete();

        InternalChild = new SpriteText
        {
            Text = FetchedStore.GetAwesomeThing()
        };
```

```
    }
}

/// <summary>
/// A class which is to be cached via DI.
/// </summary>
public class MyStore
{
    public string GetAwesomeThing() => "awesome thing!";
}
```

Members marked with either of these attributes are cached or resolved in their respective classes before the [BackgroundDependencyLoader]-annotated method is run.

Using [BackgroundDependencyLoader] to resolve

This can be useful if you want to ensure everything happens in the (potentially asynchronous) `load()` method.

```
/// <summary>
/// A class which caches something for use by children.
/// </summary>
public partial class MyGame : Game
{
    [Cached]
    protected readonly MyStore Store = new MyStore();

    public MyGame()
    {
        Child = new Container
        {
            RelativeSizeAxes = Axes.Both,
            Child = new Container
            {
                RelativeSizeAxes = Axes.Both,
                Child = new Container
                {
                    RelativeSizeAxes = Axes.Both,
                    Child = new MyComponent()
                }
            },
        };
    }
}
```

```
/// <summary>
/// A component that consumed the cached class.
/// </summary>
public partial class MyComponent : CompositeDrawable
{
    [BackgroundDependencyLoader]
    private void load(MyStore store)
    {
        InternalChild = new SpriteText
        {
            Text = store.GetAwesomeThing()
        };
    }
}

/// <summary>
/// An class which is to be cached via DI.
/// </summary>
public class MyStore
{
    public string GetAwesomeThing() => "awesome thing!";
}
```

Using `CreateChildDependencies()` to cache

Some more advanced scenarios may require use of this method instead of the `[Cached]` attribute, such as if late initialisation of the cacheable objects is required.

```
/// <summary>
/// A class which caches something for use by children.
/// </summary>
public partial class MyGame : Game
{
    protected MyStore Store;

    public MyGame()
    {
        Child = new Container
        {
            RelativeSizeAxes = Axes.Both,
            Child = new Container
            {
                RelativeSizeAxes = Axes.Both,
                Child = new Container
                {
```

```
                RelativeSizeAxes = Axes.Both,
                Child = new MyComponent()
            }
        },
    };
}

protected override IReadOnlyDependencyContainer
CreateChildDependencies(IReadOnlyDependencyContainer parent)
{
    var dependencies = new DependencyContainer(base.CreateChildDependencies(parent));
    dependencies.Cache(Store = new MyStore());
    return dependencies;
}
}
```

Note that the `DependencyContainer` class exposes two methods for caching dependencies:

- `.Cache()` will always cache the dependency using its *runtime, most derived type*. The implications of this are demonstrated by the following example:

```
public abstract class BaseDependency { }
public class DerivedDependency : BaseDependency { }

public partial class DependencyProvider
{
    private BaseDependency dependency;

    protected override IReadOnlyDependencyContainer
CreateChildDependencies(IReadOnlyDependencyContainer parent)
    {
        var dependencies = new
DependencyContainer(base.CreateChildDependencies(parent));
        dependencies.Cache(dependency = new DerivedDependency());
        return dependencies;
    }
}

public partial class DependencyConsumer
{
    [Resolved]
    private BaseDependency baseDependency { get; set; }       // WRONG - will fail
at runtime

    [Resolved]
```

```
    private DerivedDependency derivedDependency { get; set; } // OK
}
```

- `.CacheAs<T>()` will cache the dependency using its *declared* type, as demonstrated by the following example:

```
public abstract class BaseDependency { }
public class DerivedDependency : BaseDependency { }

public partial class DependencyProvider
{
    private BaseDependency dependency;

    protected override IReadOnlyDependencyContainer
CreateChildDependencies(IReadOnlyDependencyContainer parent)
    {
        var dependencies = new
DependencyContainer(base.CreateChildDependencies(parent));
        dependencies.CacheAs(dependency = new DerivedDependency());
        return dependencies;
    }
}

public partial class DependencyConsumer
{
    [Resolved]
    private BaseDependency baseDependency { get; set; }       // OK

    [Resolved]
    private DerivedDependency derivedDependency { get; set; } // WRONG - will fail
at runtime
}
```

`[Cached]` on drawable classes

Drawable classes themselves can be annotated with the `[Cached]` attribute. In that case, the attribute is interpreted such that all instances of the drawable class will cache themselves to all of their children.

The caching will use the type *at the point of declaration*. To illustrate, given the following structure:

```
[Cached]
public partial class A : Drawable { }

public partial class B : A { }
```

the following things will happen:

- Instances of `A` will cache themselves to their children using type `A`.
- Instances of `B` will cache themselves to their children using type `A`.
- Instances of `B` will **not** cache themselves to their children using type `B`. For that to happen, the `[Cached]` attribute would have to be repeated on type `B`.

`[Cached]` on interfaces implemented by a drawable class

A variant of type-based caching above is available via interfaces. Interfaces can be annotated with `[Cached]`; every `Drawable` will cache itself to its children using every interface type annotated with `[Cached]` that it implements. As an example:

```
[Cached]
public interface IFirstInterface { }

[Cached]
public interface ISecondInterface { }

public interface IThirdInterface : ISecondInterface { }

public partial class Dependency : Drawable, IFirstInterface, IThirdInterface { }
```

all instances of `Dependency`:

- will cache themselves to children as `IFirstInterface`,
- will cache themselves to children as `ISecondInterface` (transitively via `IThirdInterface`),
- will **not** cache themselves to children as `IThirdInterface` (as, analogously to classes, `[Cached]` is only valid on types it is explicitly put on)

Playing Audio

Tracks

Tracks are for playing single-instanced long-form audio. They offer the widest variety of adjustments possible, but are expensive and not recommended to be used for quick-fire or concurrent audio playback.

- **`ITrackStore`**: Used to retrieve `Track` objects. The global track store is accessible through either `AudioManager.Tracks` or by resolving an `ITrackStore` dependency into the `Drawable` object.
- **`Track`**: The track, which is usually stored at a somewhat global level and played as required.

The global track store provides access to all `.mp3` files placed in the application's `Resources/Tracks/` directory.

The following is a typical usage pattern:

```
private Track track;
private DrawableTrack drawableTrack;

[BackgroundDependencyLoader]
private void load(ITrackStore tracks)
{
    // Option 1: Raw tracks.
    // This only has the audio adjustments (e.g. volume) of the track store applied to it.
    track = tracks.Get("test-track.mp3");

    // Option 2: Drawable tracks.
    // Along with the track store's own adjustments, this version also has the adjustments
of any parenting AudioContainers applied to it.
    drawableTrack = new DrawableTrack(tracks.Get("test-sample.mp3"));
    Child = drawableTrack;

    // Typically, either of the above are stored in a field in the Drawable object to be
used later on. For example, in OnClick().

    // You can adjust the volume of all tracks retrieved from the track store. Note that
this is a global property.
    // E.g. The track playback volume is calculated as: (track_store_volume)
* (track_volume).
    tracks.Volume.Value = 0.8;

    // You can also adjust other properties on the track itself.
    track.Volume.Value = 0.8;
}
```

```
// The following examples demonstrate usage of the raw Track object, but DrawableTrack can
be used interchangeably.

private bool playing;

protected override bool OnClick(ClickEvent e)
{
    if (!playing)
        track.Start();
    else
        track.Stop();

    playing = !playing;
    return true;
}

protected override bool OnDoubleClick(DoubleClickEvent e)
{
    track.Seek(0);
    return true;
}

protected override void Dispose(bool isDisposing)
{
    base.Dispose(isDisposing);

    // Be sure to dispose the track, otherwise memory will be leaked!
    // This is automatic for DrawableTrack.
    track.Dispose();
}
```

Virtual tracks

`ITrackStore.GetVirtual()` can be used as a sane fallback value for cases where a non-null `Track` is always required. It mimics all positional methods of a non-virtual track without playing audio.

Custom track stores

In order to retrieve tracks from a different location than the default `Resources/Tracks/` directory, a custom track store is required.

To create a custom track store, pass a `ResourceStore` to `AudioManager.GetTrackStore()`. This will return an `ITrackStore` to be used for future lookups. The custom track store will have its audio (e.g. volume) adjusted by the global track store.

```
[BackgroundDependencyLoader]
private void load(AudioManager audio)
{
    var trackStore = audio.GetTrackStore(new ResourceStore<byte[]>(...));
    trackStore.Get("test-track.mp3");
}
```

Samples

Samples are for fast and concurrent playback of short audio clips. They feature automatic memory management with the intention of being fired-and-forgotten, but lack some notable features of `Track` such as time tracking, seeking, tempo adjustments, and on-demand restarting of playback.

- **`ISampleStore`**: Used to retrieve `Sample` objects. The global sample store is accessible through either `AudioManager.Samples` or by resolving an `ISampleStore` dependency into the `Drawable` object.
- **`Sample`**: The sample, which is typically stored by the `Drawable` object in some fashion - either as the raw `Sample` or nested inside a `DrawableSample` object. Used in order to retrieve and play one or more `SampleChannel`s.
- **`SampleChannel`**: The object which plays audio. This is typically fired-and-forgotten, with special cases in-case looping, adjusting volume/balance/frequency, or stopping audio playback is required.

The global sample store provides access to all `.wav` and `.mp3` files placed in the application's `Resources/Samples/` directory.

The following is a typical usage pattern:

```
private Sample sample;
private DrawableSample drawableSample;

[BackgroundDependencyLoader]
private void load(ISampleStore samples)
{
    // Option 1: Raw samples.
    // This only has the audio adjustments (e.g. volume) of the sample store applied to it.
    sample = samples.Get("test-sample.mp3");

    // Option 2: Drawable samples.
    // Along with the sample store's own adjustments, this version also has the adjustments
of any parenting AudioContainers applied to it.
    drawableSample = new DrawableSample(samples.Get("test-sample.mp3"));
    Child = drawableSample;

    // Typically, either of the above are stored in a field in the Drawable object to be
```

```
used later on. For example, in OnClick().

    // You can adjust the volume of all samples retrieved from the sample store. Note that
this is a global property.
    // E.g. The channel playback volume is calculated as: (sample_store_volume) *
(sample_volume) * (channel_volume).
    samples.Volume.Value = 0.8;
}

protected override bool OnClick(ClickEvent e)
{
    // The following examples demonstrate usage of the raw Sample object, but DrawableSample
can be used interchangeably.

    // Play a normal channel from the sample.
    // This can be called as many times as desired, but concurrent playback will only
be heard
    // up to sample.PlaybackConcurrency times from the same sample name (e.g.
"test-sample.mp3").
    sample.Play();

    // Play another normal channel. We'll be using this one for more examples later on.
    var channel2 = sample.Play();

    // Play a looping channel.
    // It's recommended to retrieve the channel to set the looping flag first, and then to
play the channel afterwards.
    // The same applies to adjusting other channel properties, e.g. setting an
initial volume.
    var channel1 = sample.GetChannel();
    channel1.Looping = true;
    channel1.Play();

    // Apply a volume adjustment to all channels (all three above, and any future ones) from
the sample.
    sample.Volume.Value = 0.5;

    // Apply an independent volume adjustment to channel 1 only.
    // The final volume of this channel will be 0.8 (store) * 0.5 (sample) * 0.5 (channel)
= 0.2.
    channel1.Volume.Value = 0.5;

    // Stop channel 2, but keep channel 1 playing.
    channel2.Stop();

    // Resume playback of channel 2. This doesn't restart unless playback has finished.
```

```
        channel2.Play();

        return true;
    }

    protected override void Dispose(bool isDisposing)
    {
        base.Dispose(isDisposing);

        // Halts playback of all played channels from the sample. This is automatic
    for DrawableSample.
        // Note: Omission doesn't always result in a memory leak, however it's recommended
    to stop
        // longer sample playback when Drawables are disposed, and especially so for
    looping channels.
        sample.Dispose();
    }
```

Virtual samples

`SampleVirtual` can be used as a sane fallback value for cases where a non-null `Sample` is always required.

Custom sample stores

In order to retrieve samples from a different location than the default `Resources/Samples/` directory, a custom sample store is required.

To create a custom sample store, pass a `ResourceStore` to `AudioManager.GetSampleStore()`. This will return an `ISampleStore` to be used for future lookups. The custom sample store will have its audio (e.g. volume) adjusted by the global sample store.

```
    [BackgroundDependencyLoader]
    private void load(AudioManager audio)
    {
        var sampleStore = audio.GetSampleStore(new ResourceStore<byte[]>(...));
        sampleStore.Get("test-sample.mp3");
    }
```

Mixing

All `SampleChannel` and `Track` audio (hereby referred to as a "channel") is routed through an `AudioMixer`. DSP effects can be applied to the `AudioMixer` to change the resultant audio of channels routed through it independent of other `AudioMixers` in the game.

Global audio mixer

The global audio mixer affects all channels by default and can be accessed through `AudioManager.Mixer`. The following is a simple example of how to apply a DSP effect globally:

```
[BackgroundDependencyLoader]
private void load(AudioManager audio, ISampleStore samples)
{
    // Add an effect to the global mixer.
    audio.Mixer.Effects.Add(new ...);

    // The sample has the above effect applied to it by default.
    samples.Get(...).Play();

    // The same is true for DrawableSample/DrawableTrack.
    DrawableSample drawableSample;
    Add(drawableSample = new DrawableSample(samples.Get(...)));
    drawableSample.Play();
}
```

Local (custom) audio mixers

The global mixer provides too wide of an effect for use in many cases. There are two ways to create a mixer for only the specific audio which needs to be affected.

One option is to create a locally managed `AudioMixer` through `AudioManager.CreateAudioMixer()`:

```
private AudioMixer mixer;

[BackgroundDependencyLoader]
private void load(AudioManager audio, ISampleStore samples)
{
    // Create the custom mixer.
    mixer = audio.CreateAudioMixer();

    // Add an effect to the mixer.
    mixer.Effects.Add(new ...);

    Sample sample = samples.Get(...);

    // Add channels to the custom mixer in order to apply the effect to them.
    SampleChannel channel = sample.GetChannel();
    mixer.Add(channel);
    channel.Play();
```

```
    // Channels that are not added to the custom mixer will only be affected by the
global mixer.
    sample.Play();
}

protected override void Dispose(bool isDisposing)
{
    base.Dispose(isDisposing);

    // Be sure to dispose the mixer, otherwise memory will be leaked!
    mixer?.Dispose();
}
```

The other option is to use `DrawableAudioMixer`, which takes care of the hard work behind the scenes:

```
[BackgroundDependencyLoader]
private void load(ISampleStore samples)
{
    DrawableAudioMixer drawableAudioMixer;
    DrawableSample drawableSample;

    // Add a mixer to the hierarchy, this will affect DrawableSample/DrawableTrack children
added to it, recursing until another DrawableAudioMixer is reached.
    // Note: It will not affect Sample/SampleChannel/Tracks unless they're added manually!
    Add(drawableAudioMixer = new DrawableAudioMixer
    {
        Child = drawableSample = new DrawableSample(samples.Get(...)),
        Effects =
        {
            ...
        }
    });

    // This sample has the above effect applied to it.
    drawableSample.Play();

    // As above, you can still add channels to the mixer manually to apply the effect
to them.
    Sample sample = samples.Get(...);
    SampleChannel channel = sample.GetChannel();
    drawableAudioMixer.Add(channel);
    channel.Play();
}
```

Effect prioritisation

Effects are applied in the order that they appear in the `AudioMixer.Effects` or `DrawableAudioMixer.Effects` list. Effects can be added, removed, or moved around in order to change their priority.

Changing the audio mixer

Channels can be moved to another audio mixer by adding or removing them from the `AudioMixer`:

```
[BackgroundDependencyLoader]
private void load(ISampleStore samples)
{
    DrawableAudioMixer mixer1;
    DrawableAudioMixer mixer2;

    Children = new Drawable[]
    {
        mixer1 = new DrawableAudioMixer(),
        mixer2 = new DrawableAudioMixer()
    };

    // Add the channel to the first mixer.
    SampleChannel channel;
    mixer1.Add(channel = samples.Get(...).Play());

    // Move the channel to the second mixer.
    mixer2.Add(channel);

    // Remove the channel from the second mixer.
    // Note: The channel will be moved to the global mixer, not mixer1 from above!
    mixer2.Remove(channel);

    // This has no effect. All channels must be played by one mixer.
    audio.Mixer.Remove(channel);



    // Similarly, for DrawableSample:
    DrawableSample drawableSample = new DrawableSample(samples.Get(.));
    SampleChannel channel1 = drawableSample.Play();
    SampleChannel channel2 = drawableSample.Play();

    // Add the entire sample to the first mixer, this affects both played channels.
    mixer1.Add(drawableSample);
```

```
    // Move the entire sample to the second mixer, this affects both played channels.
    mixer1.Remove(drawableSample);
    mixer2.Add(drawableSample);

    // Move the first channel to the first mixer.
    mixer1.Add(channel1);

    // Move the second channel to the global mixer.
    mixer2.Remove(channel2);

    // Move the entire sample to the first mixer, this still affects both played channels.
    mixer2.Remove(drawableSample);
    mixer1.Add(drawableSample);
}
```

Screens and Screen Stacks

osu!framework utilizes and implements a `Screen` concept. Screens let us specify single "views" that can be stacked on with other screens, or exited to reveal screens stacked underneath.

Visually, imagine stacking one sheet of paper on top of another. The forward facing sheet is shown to the user, while other screens remain underneath it ready to be resumed.

Only one screen can be active at a time within a single screen stack. However, multiple different screen stacks may exist to layer screens on top of one another.

Creating a screen stack

Simply add a new instance of a screen stack into your draw hierarchy.

```
private ScreenStack awesomeScreenStack = null!;

[BackgroundDependencyLoader]
private void load()
{
    Add(awesomeScreenStack = new ScreenStack());
}
```

Creating a new screen

When a screen stack is first initialized, it will be empty. We will need to create a screen before it has anything that can be shown to the user.

`Screen`s can be populated with content by adding `Drawable`s to it, like is the case with any other `CompositeDrawable`.

```
public partial class AwesomeScreen : Screen
{
    public AwesomeScreen()
    {
        AddInternal(new Box
        {
            Colour = Color4.Tomato,
            Origin = Anchor.Centre,
            Anchor = Anchor.Centre
        });
    }
}
```

If this screen is the first screen you are pushing to the stack, you should push it to the stack directly using `ScreenStack.Push()`. If you already have a screen in the stack, you can also push to the screen using `Screen.Push()`.

Handling screen transitions

Screens contain the following methods that are invoked whenever a screen change involving it would occur:

- `OnEntering(ScreenTransitionEvent)` - Invoked when the screen is added to the top of the stack using `Screen.Push()`.
- `OnExiting(ScreenExitEvent)` - Invoked when the screen is exited for the screen under it using `Screen.Exit()`. The return value can be used to cancel the exit process, which is useful in flows like showing confirmation dialogs to the user.
- `OnResuming(ScreenTransitionEvent)` - Invoked when the screen is made the active screen as a result of the current screen being exited.
- `OnSuspending(ScreenTransitionEvent)` - Invoked when another screen is pushed to the top of the stack, replacing this one.

Example

We can take care of inwards screen related transitions inside the screen we're transitioning into using the `OnEntering()` and `OnResuming()` methods. The `FadeInFromZero()` method is great for fading in a new screen, whereas `FadeIn()` would let us adjust the fade from any alpha the screen may already be at.

```
public override void OnEntering(ScreenTransitionEvent e)
{
    this.FadeInFromZero(500, Easing.OutQuint);
}

public override void OnSuspending(ScreenTransitionEvent e)
{
    this.FadeIn(500, Easing.OutQuint);
}
```

Transforms and Tweening

A relatively extensive toolchain exists for applying transforms to `Drawable`s. This includes not only the ability to perform common visual adjustments (fade, scale, rotate, colour), but also the ability to arbitrarily perform transforms on any field/property belonging to the `Drawable`.

Internally, transforms are a state machine which is build using `TransformSequence`. Multiple transforms can be chained and nested. This page attempts to provide a starting point for understanding how transforms can be used, but there should be plenty of room for exploration beyond this guide via experimentation.

Basic operations

A simple example which will fade a box into existence:

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.

box.FadeInFromZero(500);
```

Chaining

Transforms can be chained in various ways. To run multiple transforms of different types at the same time value, simply chain the calls:

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.

box.FadeInFromZero(500)
   .ScaleTo(new Vector2(2))
   .RotateTo(90); // all three of these will be run in parallel.
```

To play one transform after another finishes, use `.Then()`:

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.
```

```
box.FadeInFromZero(500).Then().FadeOut(500); // fade in then out, over 1 second.
```

Offsets and delays

To add a delay to a sequence, you can use `.Delay(ms)`:

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.

box.FadeInFromZero(500).Then().Delay(200).FadeOut(500); // fade in then out, over 1.2 seconds, with a pause at full opacity.
```

For more complex cases, `BeginDelayedSequence` and `BeginAbsoluteSequence` can help to organise things:

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.

using (box.BeginAbsoluteSequence(2000)) // nested calls will run from absolute clock time of 2000ms. by default this applies to all children too.
{
    box.FadeIn(1000);

    using (box.BeginDelayedSequence(1000)) // nested calls will be delayed by 1000ms. by default this applies to all children too.
        box.FadeOut(1000);
}
```

Looping

To loop a certain transform sequence, append either of `.Loop(millisecondsPause)`, or `.Loop(millisecondsPause, times)`, depending on how you want the loop to behave:

`.Loop(millisecondsPause)`

This will loop the transform sequence indefinitely, with a defined pause between each iteration of the loop, until it's interrupted by one of the interruption methods.

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.

// indefinitely resizes the box to 75px, then to 50px, with a pause of 250ms at the end of each iteration
box.ResizeTo(new Vector2(75), 500).Then().ResizeTo(new Vector2(50), 250).Loop(250);
```

`.Loop(millisecondsPause, times)`

Just like the above, except that it will loop for a defined number of times, not indefinitely.

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.

// for 5 times, resize the box to 75px, then to 50px, with a pause of 250ms at the end of each iteration
box.ResizeTo(new Vector2(75), 500).Then().ResizeTo(new Vector2(50), 250).Loop(250, 5);
```

Applying to arbitrary members (fields/properties)

All specific transform methods such as `.FadeIn(...)`/`.FadeColour(...)` are methods defined in extension classes that delegate to the core method `.TransformTo()`, which accepts the name of the member to transform/tween, and the remaining arguments for transforming (duration, easing, etc.).

And that doesn't apply only to the `Drawable` base class, you can apply transforms to any member belonging to your class as long as it inherits `Drawable`, and as long as the member is not `static`, and not a `readonly` field or getter-only/setter-only property:

```
public class SpecialBox : Box
{
    public double SpecialProperty { get; set }
}

var box = new SpecialBox { Size = new Vector2(50), SpecialProperty = 1.0 }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.
```

```
box.TransformTo(nameof(SpecialProperty), 10.0, 1000); // transform SpecialProperty from
current value to value 10.0, in 1 second.
```

For wrapping it in an extensions class:

```
public static class SpecialBoxExtensions
{
    public static TransformSequence<T> TweenSpecialPropertyTo<T>(this T specialBox, double
newValue, double duration, Easing easing = Easing.None)
        where T : SpecialBox
        => specialBox.TweenSpecialPropertyTo(newValue, duration, new
DefaultEasingFunction(easing));

    public static TransformSequence<T> TweenSpecialPropertyTo<T>(this TransformSequence<T>
t, double newValue, double duration, Easing easing = Easing.None)
        where T : SpecialBox
        => specialBox.TweenSpecialPropertyTo(newValue, duration, new
DefaultEasingFunction(easing));

    public static TransformSequence<T> TweenSpecialPropertyTo<T, TEasing>(this T specialBox,
double newValue, double duration, in TEasing easing)
        where T : SpecialBox
        where TEasing : IEasingFunction
        => specialBox.TransformTo(nameof(SpecialBox.SpecialProperty), newValue,
duration, easing);

    public static TransformSequence<T> TweenSpecialPropertyTo<T, TEasing>(this
TransformSequence<T> t, double newValue, double duration, in TEasing easing)
        where T : SpecialBox
        where TEasing : IEasingFunction
        => t.Append(o => o.TweenSpecialPropertyTo(newValue, duration, easing));
}


box.TweenSpecialPropertyTo(10.0, 1000);


box.FadeInFromZero(500)
   .TweenSpecialPropertyTo(10.0, 1000)
   .Then().FadeOut(500);
```

Manual interpolation

In cases where transforms are to be run every frame, it is highly recommended to use interpolation instead:

```
public override void Update()
{
    // useful if targetWidth is a moving target which need to be tracked, for example.
    box.Width = Interpolation.ValueAt(Clock.ElapsedFrameTime, box.Width, targetWidth, 0,
200, Easing.OutQuint);
}
```

Interrupting transforms

Transforms can be interrupted during their application, by finishing them immediately on current time, or removing them and leaving the transformed property at its current state, or rewinding it back to the beginning of the transform.

Finishing transforms immediately

To finish transforms immediately, use `FinishTransforms` on the drawable whose transforms are still being processed.

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before
operating on it.

box.FadeInFromZero(1000);

// after 0.5 seconds, finish transforms applied to the box.
Scheduler.AddDelay(() =>
{
    // by default, this only applies to transforms applied to the box itself,
    // pass `true` to finish transforms of children as well.
    box.FinishTransforms();
}, 500);
```

Removing transforms

To remove transforms, use `ClearTransforms` on the drawable whose transforms are still being processed.

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before
```

```
operating on it.

box.FadeInFromZero(1000);

// after 0.5 seconds, remove transforms from the box.
Scheduler.AddDelay(() =>
{
    // by default, this only applies to transforms applied to the box itself,
    // pass `true` to clear transforms of children as well.
    box.ClearTransforms();
}, 500);
```

To remove transforms starting at a certain time, use `ClearTransformsAfter` instead.

```
box.FadeInFromZero(500).Then().FadeOut(500);

// after 0.5 seconds, remove transforms at current time from the box.
Scheduler.AddDelay(() =>
{
    // by default, this only applies to transforms applied to the box itself,
    // pass `true` to clear transforms of children as well.
    box.ClearTransformsAfter(Time.Current);
}
```

- Note that `ClearTransformsAfter(time)` actually removes transforms starting at the given time, not after it. This was a mistake in naming and will be resolved soon.

Applying transforms to the drawable from a different time before removing

Removing transforms will not revert them, the drawable will remain at its current state.

To revert the transform applied to the `Drawable`, or apply them from a given absolute clock time before removing, use `ApplyTransformsAt`.

```
var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before
operating on it.

box.FadeInFromZero(1000);

// after 0.5 seconds, revert the transform and remove it from the box.
Scheduler.AddDelay(() =>
```

```
{
    // by default, this only applies to transforms applied to the box itself,
    // pass `true` to clear transforms of children as well.
    box.ApplyTransformsAt(Time.Current - 500);
    box.ClearTransforms();
}, 500);
```

Rewinding support

// TODO

Using custom easing functions

In cases the default `Easing` functions aren't what you want for some transforms, you can define your own `IEasingFunction` and pass it to the transforms:

```
public readonly struct SpecialEasingFunction : IEasingFunction
{
    public double ApplyEasing(double time) => time;
}

var box = new Box { Size = new Vector2(50) }

Add(box); // the target of transforms must always be in the draw hierarchy and loaded before operating on it.

box.FadeInFromZero(1000, new SpecialEasingFunction());
```

- `IEasingFunction.ApplyEasing` accepts a time value in the range [0-1], and returns an eased value of it. For examples, you can refer to the `DefaultEasingFunction` implementation.

Best practices

Some basic things to note:

- Transforming before a drawable is loaded (ie. `LoadComplete`) will cause the transforms to play out immediately. This is due to the drawable not yet having a clock to work with. If you must queue transforms before load, make sure to use a `Schedule(() => {})` call.
- Transforms are not free and should not be run every frame. Please use interpolation for such cases.
- Generally, we recommend not operating on oneself with transforms (ie. `this.FadeIn()`). This is due to the potential of conflicts between internal and external calls, which could overwrite or cause unexpected behaviour. Create a private nested container and operate on that instead.

Coordinate and Layout System

Once added to a parent `Container` hierarchy, `Drawable` objects will adopt the pixel coordinate space of that container. By default, all positioning and sizing is static via absolute pixel values regardless of the container size. That being said, a variety of mechanisms exist to allow relative sizing in order to properly scale to different window sizes and device screens.

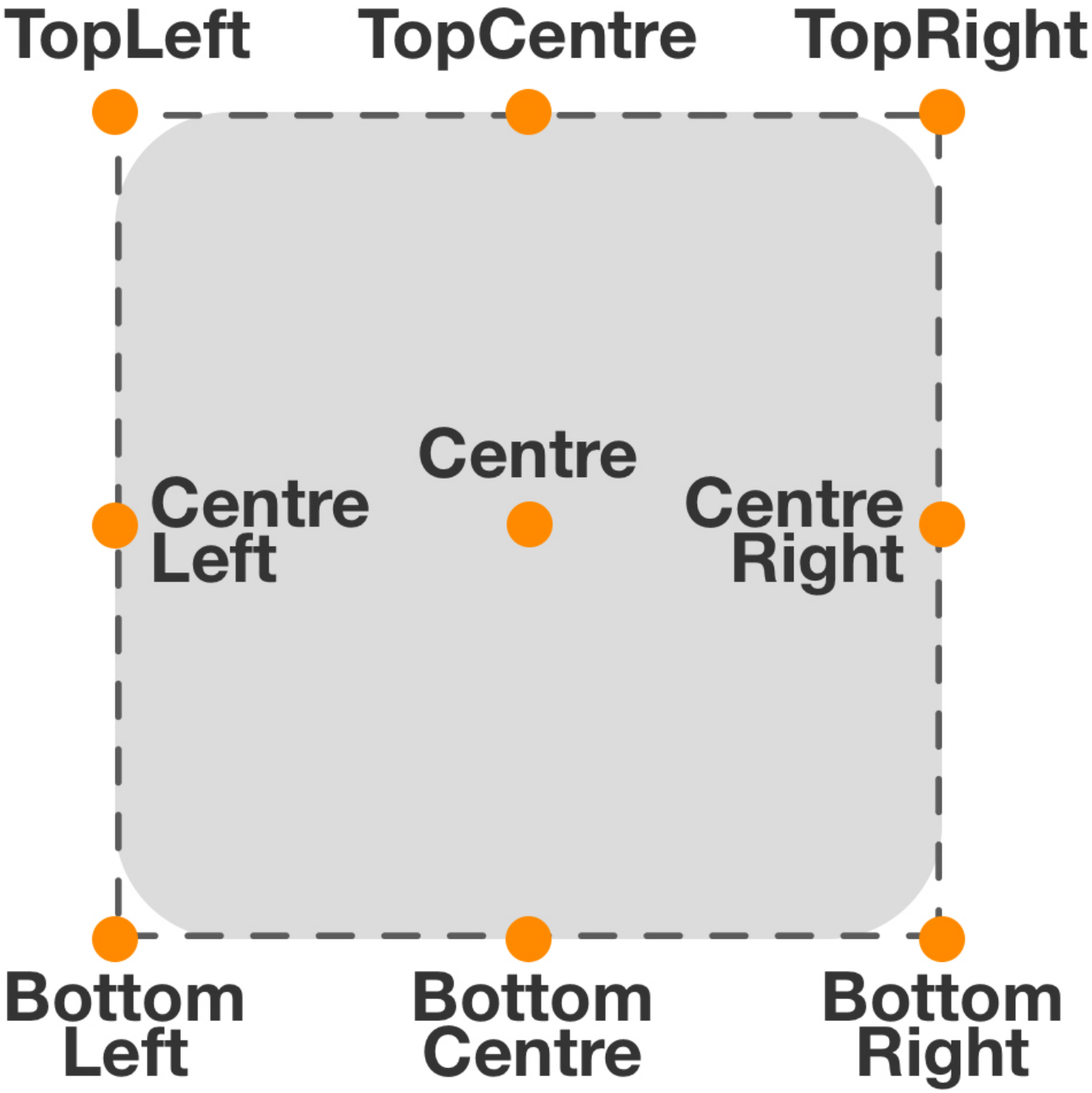
In order to help automate laying elements out in common positions, `Drawable` implements 2 properties that will help determine its initial position.

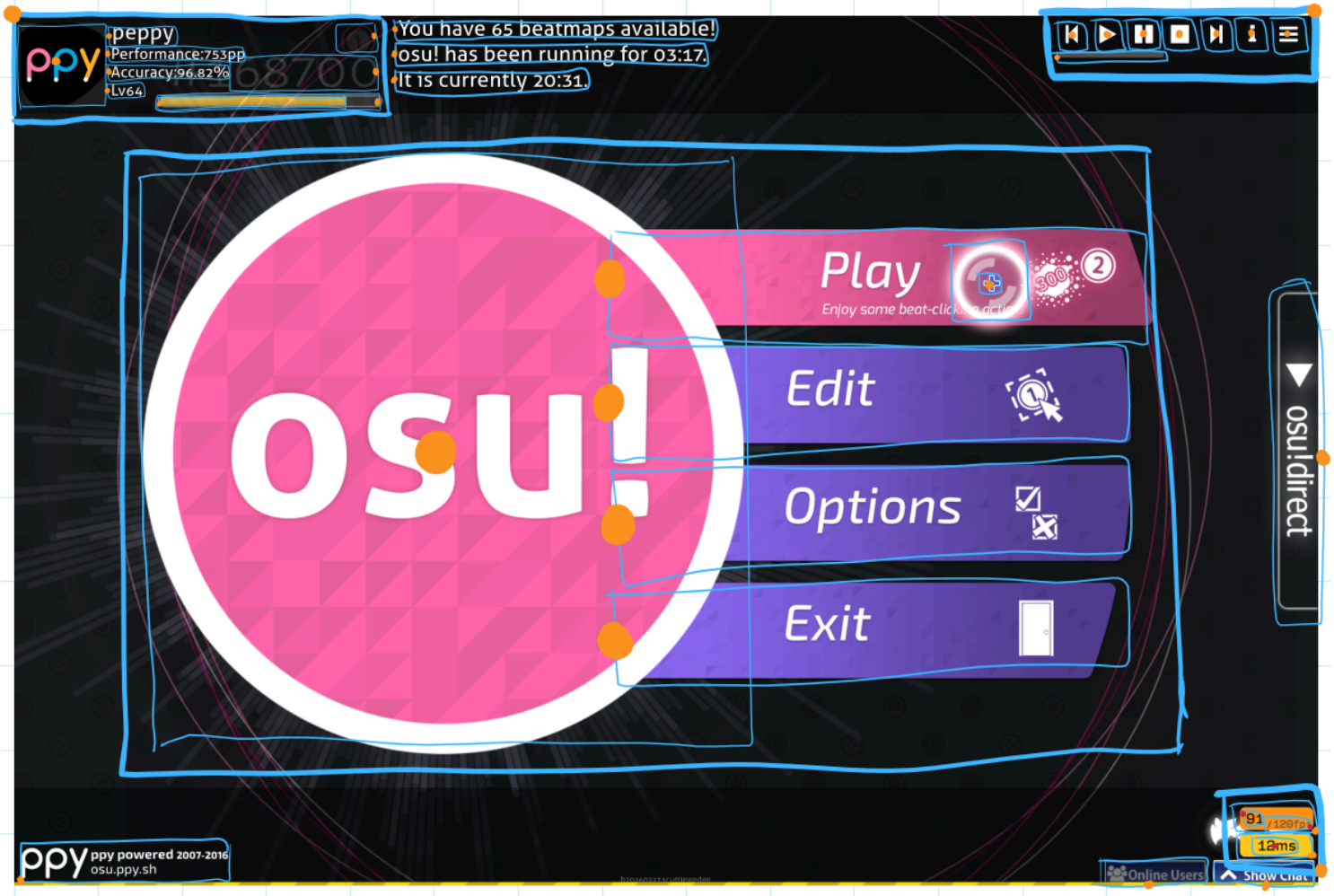
Anchor

`Anchor` will determine its origin's position relative to the parent container. `Centre` will place the element in the middle of the container, whereas other options, such as `TopLeft` will position it along the outer edges of the container.

Origin

`Origin` will determine the offset of the element's content from its own origin. For example `Centre` will mean the origin of the element will be located in the middle of its content. `TopLeft` will make the origin the top left corner of the content.


TopLeft
TopCentre
TopRight
Centre
Centre Left
Centre Right
Bottom Left
Bottom Centre
Bottom Right



Once these two properties have been set, they can be further customized with additional properties.

- Setting `Position` on a `Drawable` will offset it from its initial position. This supports both positive and negative values to offset the element in any direction.
- Setting `Size` will change the size, in pixels of the width and height of the element. The positioning of the element will be updated around its `Anchor` and `Origin` values factoring in its new size.

```
box.Position = new Vector2(100.0f, 50.0f);
box.Size = new Vector2(400.0f, 400.0f);
```

Relative Layout and Sizing

For cases where the size of the container is flexible, it is possible to override the behavior of `Drawable` to instead rely on relative values, instead of absolute.

To enable this behaviour, access the `RelativePositionAxes` and `RelativeSizeAxes` enum properties, and set which axes you would to be relative.

Once enabled, instead of absolute pixel values, you can set values between `0.0f` and `1.0f` to denote the relative value. In this case `0.0f` represents 0% of the parent container, and `1.0f` represents 100%.

```
box.RelativePositionAxes = Axes.Both;
box.Position = new Vector2(0.2f, 0.5f);
```

Scaling Absolute Values to Screen Size

In most cases, working with relative values is much more vague and difficult to work in as opposed to absolute values.

In order to allow absolute positioning whilst still allowing flexible screen sizes, osu-framework provides a type of `Container` called `DrawSizePreservingFillContainer`.

```
new DrawSizePreservingFillContainer
{
    Child = box = new Box
    {
        Anchor = Anchor.TopLeft,
        Origin = Anchor.TopLeft,
        Colour = Color4.Orange,
        Size = new Vector2(200),
    }
};
```

This type of container implements a `TargetDrawSize` property (Default value is `Vector2(1024, 768)`) which represents the size of its canvas. All children can then be sized in relation to this container size.

When the parent container is resized, `DrawSizePreservingFillContainer` will resize to fill that parent, but will also properly upscale or downscale all of the child `Drawable` elements to fill the same space.

By default, `DrawSizePreservingFillContainer` will scale all of its child content while preserving the content's aspect ratio using a 'scale to fit' strategy. Depending on your content, and the size of the screens you plan to support, the `Strategy` enum allows for other types of scaling as well:

- `Minimum` - The aspect ratio scale is determined by the smaller dimension value of the container's current size. This results in all of the content being scaled down to fit into the container with no overflow. This is the default value
- `Maximum` - The aspect ratio scale is based off the larger dimension value of the container's current size. This results in content being scaled up to fill the available boundaries and *may* result in some content getting clipped.
- `Average` - The average value between the minimum and the maximum scales is calculated and applied. This is a good compromise between the both previous values.
- `Separate` - No aspect ratio is applied at all. All elements will stretch and distort relative to the container's dimensions.

Localisation

LocalisableString

Localisation in `osu-framework` revolves around the `LocalisableString` class, which UI controls and text destinations accept in place of `string`. This page explains the provided methods of interacting with `LocalisableString`, but further customisation is also possible by implementing `ILocalisableStringData` directly.

LocalisableFormattableString

Represents a string which can be formatted based on the currently selected locale, exposed in two methods and one extension method:

- `LocalisableString.Format`, accepting a format string and a list of arguments designed in a similar fashion to `string.Format`:

```
// Text = string.Format("{0} + {1} = {2}", localisable1, localisable2, localisable3);
Text = LocalisableString.Format("{0} + {1} = {2}", localisable1,
localisable2, localisable3);
```

- `LocalisableString.Interpolate`, accepting an interpolated string (`$"..."`):

```
// Text = $"{localisable1} + {localisable2} = {localisable3}";
Text = LocalisableString.Interpolate($"{localisable1} + {localisable2}
= {localisable3}");
```

- `LocalisableStringExtensions.ToLocalisableString`, accepting an `IFormattable` object (e.g. number types, `DateTime`s, `TimeSpan`s, etc.) and a format string, in a similar fashion to `IFomrattable.ToString`:

```
// Text = formattable.ToString("N0");
Text = formattable.ToLocalisableString("N0");
```

TranslatableString

Represents a string which can be translated across different languages via a specific "key" that is used to look up on the `ILocalisationStore` associated with the locale.

```
Text = new TranslatableString("music_string_key", "music");
```

```
string Get(string lookup) // ILocalisationStore.Get(string)
{
    if (lookup == "music_string_key")
    {
        switch (EffectiveCulture.Name)
        {
            case "en": return "music";
            case "ja": return "音楽";
            ...
        }
    }
}
```

In addition, `TranslatableString`s support formatting, allowing for the translated strings to specify placeholder items which are replaced with the given arguments list, in a similar fashion to `string.Format`/`LocalisableString.Format`.

```
Text = new TranslatableString("music_string_key", "{0} music",
number.ToLocalisableString("0.00%"));

string Get(string lookup) // ILocalisationStore.Get(string)
{
    if (lookup == "music_string_key")
    {
        switch (EffectiveCulture.Name)
        {
            case "en": return "{0} music";
            case "ja": return "音楽 {0}";
            ...
        }
    }
}
```

RomanisableString

Represents a unicode string that has a romanised variant, in which can be toggled on/off by the `FrameworkSetting.ShowUnicode` setting available in the config manager.

```
Text = new RomanisableString("音楽", "ongaku");
// FrameworkSetting.ShowUnicode == true: "音楽" is shown
// FrameworkSetting.ShowUnicode == false: "ongaku" is shown
```

CaseTransformableString

Represents a string which accepts a `LocalisableString` and transforms it to the specified casing, exposed in a set of extension methods which accept either `LocalisableString` or `ILocalisableStringData` (string implementations):

- `LocalisableStringExtensions.ToUpper`, which converts all characters of the input string to uppercase, using `TextInfo.ToUpper`.
- `LocalisableStringExtensions.ToLower`, which converts all characters of the input string to lowercase, using `TextInfo.ToLower`.
- `LocalisableStringExtensions.ToTitle`, which converts the first character of every word to uppercase while the rest to lowercase, using `TextInfo.ToTitleCase`.

It's implemented in a way that allows it to build itself over a `LocalisableString` or any string implementation:

```
Text = LocalisableString.Format($"{localisable1} -> {localisable2}").ToUpper();
Text = new RomanisableString("音楽", "ongaku").ToTitle();
...
```

Custom string implementations

`LocalisableString` accepts any implementation that confronts to the `ILocalisableStringData` interface, in which it can retrieve the final localised string with.

```
    public class CustomLocalisableString : IEquatable<CustomLocalisableString>,
ILocalisableStringData
    {
        public string GetLocalised(LocalisationParameters parameters)
        {
            return /* localisation logic */;
        }

        public bool Equals(CustomLocalisableString other)
        {
            if (ReferenceEquals(null, other)) return false;
            if (ReferenceEquals(this, other)) return true;

            return /* equality */;
        }

        public bool Equals(ILocalisableStringData other) => other is CustomLocalisableString
custom && Equals(custom);
        public override bool Equals(object? obj) => obj is CustomLocalisableString custom
&& Equals(custom);
```

```
        public override int GetHashCode() => /* hash code */;
    }
```

LocalisationParameters

String implementations require context to be able to localise the string. `LocalisationParameters` provide enough context to fit the need for the framework's own string implementations, but it can be built upon for custom string implementations.

This can be achieved by creating a `LocalisationManager` subclass that overrides the `LocalisationParameters` factory method and calls `UpdateLocalisationParameters` to update the parameters:

```
    public class CustomLocalisationManager : LocalisationManager
    {
        private readonly IBindable<int> custom = new BindableInt();

        public CustomLocalisationManager(FrameworkConfigManager frameworkConfig,
CustomConfigManager config)
            : base(frameworkConfig)
        {
            config.BindWith(CustomSetting.Custom, custom);
            custom.BindValueChanged(_ => UpdateLocalisationParameters(), true);
        }

        protected override LocalisationParameters CreateLocalisationParameters() => new
CustomLocalisationParameters(base.CreateLocalisationParameters(), custom.Value);

        protected class CustomLocalisationParameters : LocalisationParameters
        {
            public readonly int Custom;

            protected CustomLocalisationParameters(CustomLocalisationParameters parameters)
                : base(parameters, parameters.Custom)
            {
            }

            public CustomLocalisationParameters(LocalisationParameters parameters,
int custom)
                : base(parameters)
            {
                Custom = custom;
            }
```

```
        }
    }


        public string GetLocalised(LocalisationParameters parameters)
        {
            var customParameters = (CustomLocalisationParameters)parameters;
            return /* localisation logic */;
        }
```

LocalisableDescriptionAttribute

Represents an attribute for assigning localised description strings to classes/enums, similar to `DescriptionAttribute`.

Since attributes in general only allow constant input, the `LocalisableString`s to use for the attribute must be stored statically in a public field or a property:

```
public static class Strings
{
    public static LocalisableString Class => ...;
    public static LocalisableString EnumOne => ...;
    public static LocalisableString EnumTwo => ...;
    public static LocalisableString EnumThree => ...;
}
```

The way `LocalisableDescriptionAttribute` works is by specifying the `Type` of the class containing the `LocalisableString` member, followed by the name of said member for lookup:

```
// [Description("Class")]
[LocalisableDescription(typeof(Strings), nameof(Strings.Class))]
public class ClassWithLocalisableDescription
{
}

public enum EnumWithLocalisableDescription
{
    // [Description("One")]
    [LocalisableDescription(typeof(Strings), nameof(Strings.EnumOne))]
    One,

    // [Description("Two")]
    [LocalisableDescription(typeof(Strings), nameof(Strings.EnumTwo))]
```

```
    Two,

    // [Description("Three")]
    [LocalisableDescription(typeof(Strings), nameof(Strings.EnumThree))]
    Three,
}
```

And in order to retrieve the description, use the extension method `ExtensionMethods.GetLocalisableDescription`:

```
// Text = classInstance.GetDescription();
Text = classInstance.GetLocalisableDescription();
// Text = enumValue.GetDescription();
Text = enumValue.GetLocalisableDescription();
```

OSU_TESTS_FORCED_GC

When running tests in a grouped fashion, unobserved exceptions will be thrown at the next garbage collection (as they are handled in the finalizer of related `Task`s). This can make it very hard to track down issues stemming from unobserved exceptions. Setting this variable to `1` will force a garbage collection after each test method, increasing the chances of unobserved exceptions firing against their related tests.

Note that this adds a considerable overhead to test runs, especially on projects with large allocations. For osu! we see an overhead of around 80-100%. For this reason, it is disabled by default.

OSU_TESTS_NO_TIMEOUT

By default, `AddUntilStep` has a timeout to ensure tests don't run forever. Settings this variable to `1` will disable this timeout. This is useful when diagnosing deadlock scenarios, or to allow ample time to attach a debugger and check on the current state of the game in a fail case.

OSU_EXECUTION_MODE

Force an execution mode for test runs. Valid values are `SingleThread` and `MultiThreaded`. Default is `MultiThreaded`.

OSU_GRAPHICS_RENDERER

Selects the graphics renderer implementation. Valid values are:

- `gl` or `opengl` - the `GLRenderer` implementation.
  - Only supports the `opengl` graphics surface.
- `veldrid` - the `VeldridRenderer` implementation.

OSU_GRAPHICS_SURFACE

Selects the graphics surface that the renderer should use. Valid values are:

- `opengl`
- `metal` (only on Apple operating systems)
- `direct3d11` (only on Windows operating systems)
- `vulkan` (except Apple operating systems)

OSU_GRAPHICS_NO_SSBO

Enables/disables use of Shader Storage Buffer Objects. When disabled, the game falls back to Uniform Buffer Objects that are more limited in functionality but more broadly supported.

Valid values are:

- `0` - Disabled
- `1` - Enabled

OSU_FRAME_STATISTICS_VIA_TOUCH

Whether to allow access to framework overlays via touch input. When set to `1` (default):

- Double-click on any area on the bottom right corner of the screen to cycle between frame statistics display (and also make it visible from hidden state).
- Tap with a single finger to expand the frame statistics display when in full state (similar to holding Ctrl).
- Tap with a second finger to freeze the frame statistics display (similar to holding Shift).

Note that this only works in debug builds. Disable for debug builds by setting to `0`.

OSU_INSECURE_REQUESTS

Whether to allow non-SSL web requests. When not provided, `http://` requests will be automatically converted to `https://`.

Only available in debug builds.

Common Scenarios

This page intends to contains a bunch of common scenarios which users may run into while using the framework, which may have straightforward but not immediately obvious answers. Each entry should be as brief as possible, and include a code sample where applicable.

`GridContainer` where earlier cells handle input before later cells

This can be achieved by setting `Depth` on cell content. Constructing a later cell with higher depth than a preceding one will allow it to handle input earlier.

Creating a rounded line with end caps outside the size of the filled portion

```
public class RoundedLine : CompositeDrawable
{
    private readonly Circle content;

    public override Quad ScreenSpaceDrawQuad => content.ScreenSpaceDrawQuad;

    public ExtendableCircle()
    {
        Padding = new MarginPadding { Horizontal = -circle_size / 2f };
        InternalChild = content = new Circle
        {
            RelativeSizeAxes = Axes.Both
        };
    }
}
```

Development and Testing

All initial development of (and further improvements applied) to components in a project should be developed in the VisualTests environment. There are several advantages to doing this.

Hot Reload

Test scenes are integrated with the .NET Hot Reload mechanism. Here is an example of how this looks in practice:

https://github.com/ppy/osu-framework/assets/20418176/a3598d28-5347-45a2-9ef7-43fab5895a83

Keep in mind that the Hot Reload facility is subject to some limitations and some changes will require a full application restart. For more information, see *Supported Edits in Edit & Continue (EnC)*.

Steps and automated testing

You can use the `AddStep` function to add automatic steps which should be performed. Asserts can also be added as steps. These steps can be run in an interactive environment by the user, but also run in a headless execution, allowing for CI process testing.

The types of steps available to be added are as follows:

- `AddStep()` creates a step that runs a method. Completes successfully if no exceptions are caught.
- `AddRepeatStep()` creates a step that runs a method a specified amount of times. Completes successfully if no exceptions are caught.
- `AddToggleStep()` toggles a flag.
- `AddWaitStep()` adds a step that waits a specified amount of time before continuing to the next step.
- `AddSliderStep()` adds a step that creates a slider-bar that adjusts a set value.
- `AddAssert()` creates a step that fails if the specified value does not return true.
- `AddUntilStep()` adds a step that attempts to run until a condition becomes true, or fails when it times out.

Both `AddAssert()` and `AddUntilStep()` support the NUnit assertion constraint model, i.e.:

```
AddAssert("no text is selected", () => textBox.SelectedText, () => Is.Empty);
```

Using this style is highly recommended as the NUnit matchers provide vastly improved debugging output in case of failure compared to just plain true/false assertions.

Caveats

- Hot reload has no effect if the application is started with a debugger attached. Attempting to attach debugger after launching and applying changes via hot reload will fail with error `0x80131c69` (`CORDBG_E_ASSEMBLY_UPDATES_APPLIED`).
- If the debugger is attached, `UntilStep`s will never time out. This is a convenience feature to improve debugging of the failing condition in the until step.

Testing Local Framework Checkout with Other Projects

A useful shell script to remove a nuget reference to framework and replace with a local checkout (at the same directory depth as your project).

shell:

```
CSPROJ="osu.Game/osu.Game.csproj"
SLN="osu.sln"

dotnet remove $CSPROJ package ppy.osu.Framework
dotnet sln $SLN add ../osu-framework/osu.Framework/osu.Framework.csproj ../osu-framework/osu.Framework.NativeLibs/osu.Framework.NativeLibs.csproj
dotnet add $CSPROJ reference ../osu-framework/osu.Framework/osu.Framework.csproj

SLNF="osu.Desktop.slnf"
tmp=$(mktemp)
jq '.solution.projects += ["../osu-framework/osu.Framework/osu.Framework.csproj", "../osu-framework/osu.Framework.NativeLibs/osu.Framework.NativeLibs.csproj"]' osu.Desktop.slnf > $tmp
mv -f $tmp $SLNF
```

powershell:

```
$CSPROJ="osu.Game/osu.Game.csproj"
$SLN="osu.sln"

dotnet remove $CSPROJ package ppy.osu.Framework;
dotnet sln $SLN add ../osu-framework/osu.Framework/osu.Framework.csproj ../osu-framework/osu.Framework.NativeLibs/osu.Framework.NativeLibs.csproj;
dotnet add $CSPROJ reference ../osu-framework/osu.Framework/osu.Framework.csproj

$SLNF=Get-Content "osu.Desktop.slnf" | ConvertFrom-Json
$TMP=New-TemporaryFile
$SLNF.solution.projects += ("../osu-framework/osu.Framework/osu.Framework.csproj", "../osu-framework/osu.Framework.NativeLibs/osu.Framework.NativeLibs.csproj")
ConvertTo-Json $SLNF | Out-File $TMP -Encoding UTF8
Move-Item -Path $TMP -Destination "osu.Desktop.slnf" -Force
```

in **addition**, for iOS:

shell:

```
PROPS="osu.iOS.props";
CSPROJ="osu.iOS/osu.iOS.csproj";
SLN="osu.sln";
sed -i "" -e "s/<PackageReference Include=\"ppy\.osu\.Framework\".*$/<ProjectReference
Include=\"..\/..\/osu-framework\/osu.Framework\/osu.Framework.csproj\" \/>/g" $PROPS;
sed -i "" -e "s/<PackageReference Include=\"ppy\.osu\.Framework\.iOS\".*$/<ProjectReference
Include=\"..\/..\/osu-framework\/osu.Framework.iOS\/osu.Framework.iOS.csproj\"
\/>/g" $PROPS;
dotnet sln $SLN add ../osu-framework/osu.Framework/osu.Framework.csproj ../osu-
framework/osu.Framework.iOS/osu.Framework.iOS.csproj ../osu-
framework/osu.Framework.NativeLibs/osu.Framework.NativeLibs.csproj;
```

in **addition**, for Android:

powershell:

```
$PROPS="osu.Android.props"
$SLN="osu.sln"

dotnet sln $SLN add ../osu-framework/osu.Framework.Android/osu.Framework.Android.csproj;
((Get-Content -Path $PROPS) -replace '<PackageReference
Include="ppy\.osu\.Framework\.Android" .*',"<ProjectReference Include=`"../../osu-
framework/osu.Framework/osu.Framework.csproj`" />`n    <ProjectReference
Include=`"../../osu-framework/osu.Framework.Android/osu.Framework.Android.csproj`" />") |
Set-Content -Path $PROPS
```

Threading

With regard to threading, osu!framework games can run in two modes:

- In *multi-threaded* execution mode, the game is ran on four main threads:
    1. The input thread:
        - usually runs at 1000Hz
        - interfaces with the windowing toolkit used (SDL2 by default, osuTK also available)
        - responsible for handling input events and forwarding them to components/drawables
    2. The audio thread:
        - usually runs at 1000Hz
        - interfaces with the audio subsystem of the OS via the BASS audio library
    3. The update thread:
        - usually runs at twice the framerate limit (which can be toggled using the `FrameSync` framework setting, or the `Ctrl` + `F7` shortcut)
        - is responsible for running update code (`UpdateSubTree()`, `Update()`) for all components in the game's drawable hierarchy
    4. The draw thread:
        - runs at the framerate limit specified
        - dispatches draw calls to the GPU (currently using OpenGL, more backends planned in the future)
- In *single-thread* execution mode, the four threads above run on one OS thread in an interleaved fashion.

The threading mode can be set using the `ExecutionMode` framework setting, or using the `Ctrl` + `Alt` + `F7` shortcut.

Note that in either mode, it is possible that other tasks, such as asynchronous loads of drawables via `CompositeDrawable.LoadComponentAsync()` (see docs on asynchronous loading), or `Task`s running on the .NET runtime's thread pool, can use additional OS threads.

Scheduling

Relatively often, there is a need to change the state of a `Drawable` or `Component` in response from a callback which is not executing on the game's update thread. An example of this would be updating the text and transforms to a `SpriteText` in response to a web request. To ensure that such an operation can be executed in a safe manner, each `Drawable` or `Component` has an internal `Scheduler`, that can collect such pending operations and execute them at the correct time:

```
request = CreateRequest();
request.Failure += exception => Schedule(() =>
```

```
{
    errorMessageSpriteText.Text = exception.Message;
    errorMessageSpriteText.FadeTo(Colour4.Red).Then().FadeTo(Colour4.White, 1000);
});
```

While the above use case is to be considered the primary one for schedulers, they can also be used to handle a variety of other tasks, as well:

- If a drawable - for whatever reason - is not being actively updated (due to being not present, masked away, not loaded yet), scheduling a task onto it ensures that the task may execute on the nearest possible occasion when the drawable is back in the active scene graph.
- The return value of a `Schedule()` call is a `ScheduledDelegate`, which can be later cancelled. This can be used to enqueue operations to be executed at some point in the future, but still be able to early-cancel them if some other condition is met.
- Aside from `Schedule()`, the `Scheduler` itself is also exposed as protected on each drawable, and provides additional capabilities, such as:
    - `Scheduler.AddOnce(task)` can be used to ensure that an operation runs at most once per frame. This is most often useful when called from an input event handler (whether mouse, keyboard, or any other peripheral), to avoid performing needless updates.
    - `Scheduler.AddDelayed(task, timeUntilRun)` can be used to run an operation at a set time in the future, relative to now. That method also has a `repeat` argument that can be used for recurring operations (they will execute regularly every `timeUntilRun` milliseconds).
- As a last-ditch attempt, schedulers can sometimes also be used to fix various one-frame issues, such as auto-size containers flickering when their contents are being changed. This sort of usage is however discouraged, as it can cause headaches down the line due to complicating interactions between components.

In general schedulers are a powerful tool, but ones that should be used carefully, as they change the usual flow of operations and therefore delay their effects. Nesting schedules, or having chains of scheduled operations will complicate debugging significantly.

Audio components

Note that when using `AudioComponent` and its derived classes (`SampleChannel`, `Track`) there is no inherent need to schedule operations like `Play()` or `Stop()`. The components themselves internally ensure that the operations to be executed are correctly enqueued onto the audio thread.

Asynchronous disposal

All drawables/components in a game, upon being removed from their parent `CompositeDrawable`s, are enqueued onto a game-global async disposal queue. The queue uses TPL threadpool threads to run each of the drawables' `Dispose()` methods.

To suppress this behaviour, methods such as `CompositeDrawable.Remove()` or `CompositeDrawable.Clear(false)` can be used to remove a drawable from the hierarchy without calling its `Dispose()` implicitly.

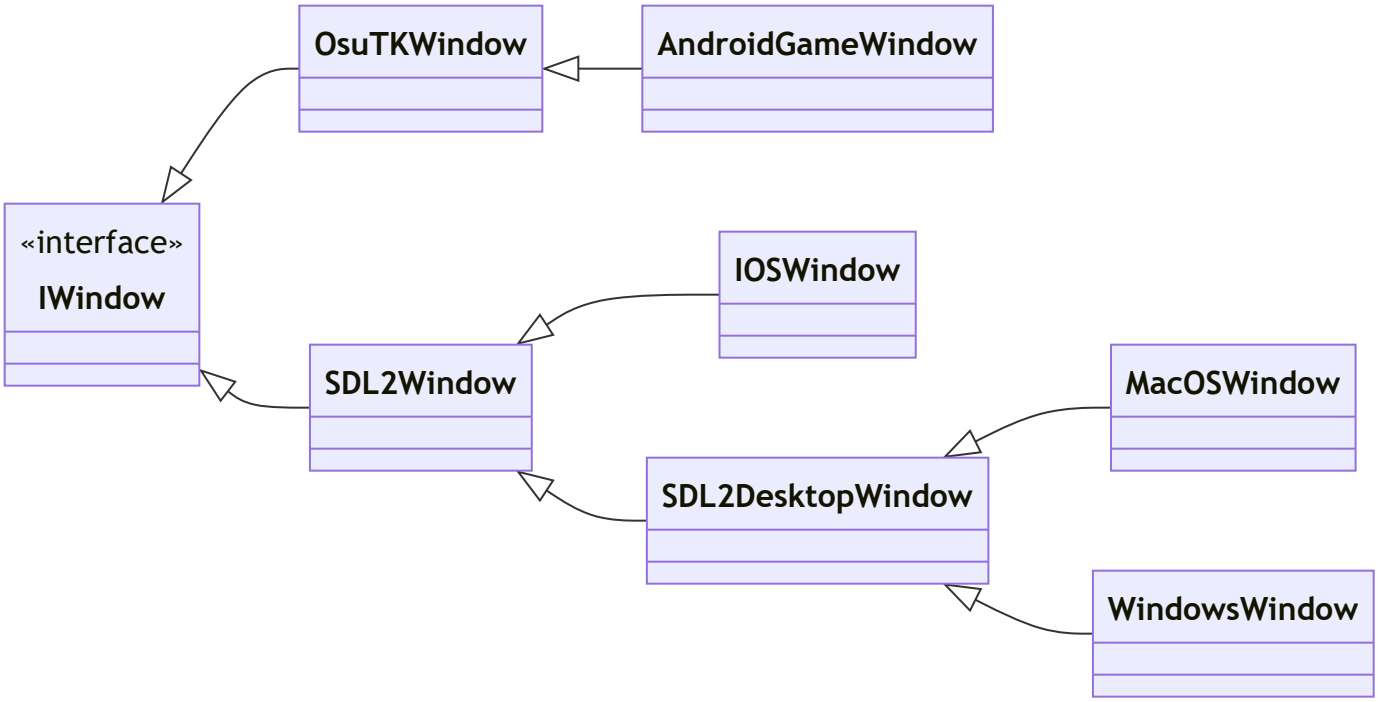
Setting thread culture

To change all of the main game threads to a new culture, you can set the `Locale` framework setting to the name of the desired locale, which should correspond to the value of `CultureInfo.Name` of the desired culture. Doing this will change culture on all of the main threads, as well as set the default culture for any new threads. Some existing threads not managed by the framework may still continue operating using the old culture, however.

Window Classes and Alternate Backends

Originally osu!framework used the OpenTK library for windowing, then transitioning to a custom fork of it (osuTK), and finally moving over to SDL2. The transition is not fully complete; the only platform remaining which is dependent on osuTK is Android.

The class hierarchy currently looks as follows:



- `IOSWindow` contains iOS-specific SDL calls to enable `CADisplayLink` to work for lower latency.
- `MacOSWindow` contains platform-specific code for precise scroll wheel handling.
- `WindowsWindow` contains platform-specific code for DPI awareness, IME, icon setting, and raw input handling.